הפקולטה להנדסה
המחלקה להנדסת חשמל ומחשבים

פרויקט גמר

קורס חשיפה לחומרה

HomeGuard

מערכת אבטחה היברידית מרובת שכבות

מחשוב קצה על בקרי ESP32, בקנד Python עם זיהוי פנים ואפליקציית שליטה לטלפון נייד

תאריך הביצוע: dd/mm/yy

תאריך ההגשה: dd/mm/yy

	שותף א'	שותף ב'
שמות המבצעים:	גיא בר	יאיר אברהם בונפיל
מספר זהות:	213263932	325534121
דוא"ל:	[להשלים]@mail.tau.ac.il	yairbonfil@mail.tau.ac.il
חתימת המגישים:		

שמות המנחים: אריאלה צברי, עידן הבר

[לאמת מול הנחיית הפרויקט]

1. תקציר

בפרויקט זה תוכננה, מומשה ונבדקה מערכת אבטחה ביתית היברידית מרובת שכבות, המשלבת מחשוב קצה על גבי בקרי ESP32, עיבוד מרכזי בסביבת Python ואפליקציית שליטה לטלפון נייד. הבעיה שנבחנה היא היעדר מערכת אזעקה זולה, אמינה ופתוחה לבקרה, המשלבת מספר שכבות הגנה: ניטור היקפי, זיהוי פריצה בכוח, בקרת כניסה פיזית וזיהוי ביומטרי של פנים, ובה בעת מאפשרת שליטה והתראות מרחוק מבלי לחשוף את המערכת לרשת האינטרנט הפתוחה.

הפתרון המוצע מבוסס על ארבעה בקרי ESP32 נפרדים: בקר חיישן אולטרה-סוני (HC-SR04) לניטור היקפי, בקר חיישן רעידות (SW-420) לזיהוי ניסיון פריצה בכוח, בקר דלת הכולל חיישן מגנטי (MC-38), קודן מטריצה 3x4 ושני זמזמים, המשמש כסמכות הבלעדית לדריכה ולנטרול, ובקר ESP32-CAM המזרים וידאו וקולט פריימים לזיהוי פנים. הבקרים מדווחים באמצעות HTTP לבקנד המבוסס FastAPI, המממש מכונת מצבים (Disarmed, Transitional, Armed, Entry Delay, Triggered), מזהה פנים באמצעות המודלים YuNet ו-SFace של ספריית OpenCV, ומשרת אפליקציית Flutter באמצעות REST ו-WebSocket. אבטחת המערכת נשענת על אימות דו-צדדי (אסימון סשן לאפליקציה ומפתח מכשיר ללוחות), שמירת סיסמה כ-hash מסוג scrypt, ונעילה לאחר שלושה ניסיונות שגויים. התראות מגיעות גם כשהאפליקציה סגורה באמצעות שירות ntfy, והגישה מרחוק מתבצעת דרך רשת פרטית מוצפנת (Tailscale).

כל דרישות ה-Must (זיהוי חדירה או פריצה בכוח, נטרול באמצעות קודן פיזי ואזעקה קולית) ודרישות ה-Nice-to-have (זיהוי פנים, יומן אירועים בזמן אמת ושינוי סיסמה מהאפליקציה) מומשו. המערכת נבדקה על חומרה אמיתית בכל ארבעת הבקרים, וכן באמצעות 258 בדיקות אוטומטיות פנימיות ו-98 בדיקות מקצה לקצה מול שרת חי, שכולן עברו. במסגרת הדוח מוצגות גם מגבלות המערכת: SFace עצמו אינו מזהה חיות, ולפיכך נוספו בקנד שתי בדיקות חיות (תנועת נקודות ציון בפנים ומודל MiniFASNetV2), אך ספי ההחלטה שלהן הם הערכות ראשוניות שטרם כוילו מול ניסיונות הונאה אמיתיים. מסיבה זו נטרול לפי פנים כבוי כברירת מחדל ומוגדר כנוחות ולא כמנגנון אבטחה.

מילות מפתח: ESP32, מערכת אבטחה, מחשוב קצה, זיהוי פנים, FastAPI, Flutter, מכונת מצבים, IoT.

2. מבוא

2.1 רקע ומוטיבציה

אבטחת הבית היא אחד מתחומי היישום הנפוצים של האינטרנט של הדברים (Internet of Things — IoT). התפתחותם של בקרי מיקרו זולים בעלי חיבור WiFi מובנה, ובראשם משפחת ה-ESP32 [1], אפשרה לבנות מערכות ניטור שבעבר דרשו חומרה ייעודית ויקרה. עם זאת, בקר מסוג זה מוגבל בכוח העיבוד ובזיכרון, ולפיכך אינו מתאים להרצת משימות כבדות כגון זיהוי פנים. מכאן נובעת הגישה ההיברידית: ביזור תפקידים, שבו בקרי הקצה אחראים על החישה והפעלה בזמן אמת, בעוד שהעיבוד הכבד, ניהול המצב והממשק למשתמש מבוצעים על מחשב מרכזי.

מערכות אזעקה מסחריות מתבססות לרוב על מנוי לענן של היצרן. גישה זו נוחה, אך מחייבת אמון בצד שלישי, תלות בזמינות השירות וחשיפה של נתוני הבית. לעומתה, מערכת הנבנית באופן עצמאי מאפשרת שליטה מלאה בנתונים ובלוגיקה, ומהווה פלטפורמה הנדסית שלמה הכוללת חומרה, קושחה, תקשורת רשת, אבטחת מידע, ראייה ממוחשבת ופיתוח אפליקציה.

2.2 הגדרת הבעיה

מערכת אזעקה ביתית אמינה צריכה לעמוד בו-זמנית במספר אתגרים הנדסיים:

1. **כיסוי מספר וקטורי תקיפה:** כניסה דרך הדלת, מעבר בהיקף וניסיון פריצה בכוח הם אירועים בעלי חתימה פיזיקלית שונה, ולפיכך דורשים חיישנים שונים.
2. **אמינות ושקיפות מצב:** כשל בלוח בודד (נפילת חשמל, ניתוק WiFi) אינו יכול להישאר סמוי. מערכת שמציגה "מנוטרל" או "מחובר" כאשר המצב האמיתי אינו ידוע גרועה ממערכת שאינה קיימת.
3. **בקרת גישה מאובטחת:** נטרול האזעקה מהקודן ומהאפליקציה הוא פעולה רגישה, וזיוף דיווח (פריצה, או "המערכת נוטרלה") עלול לגרום לכשל כפול — אזעקת שווא או הסתרת פריצה אמיתית.
4. **גישה מרחוק ללא חשיפה:** מערכת אזעקה שנחשפת לאינטרנט הפתוח חשופה לתקיפה, בעוד שמערכת הנגישה רק ברשת המקומית אינה מועילה למשתמש שנמצא מחוץ לבית.
5. **התראה בזמן ובמקום הנכון:** התראה שמגיעה רק כשהאפליקציה פתוחה אינה מספקת מענה למצב שבו האפליקציה סגורה.

2.3 מטרות הפרויקט

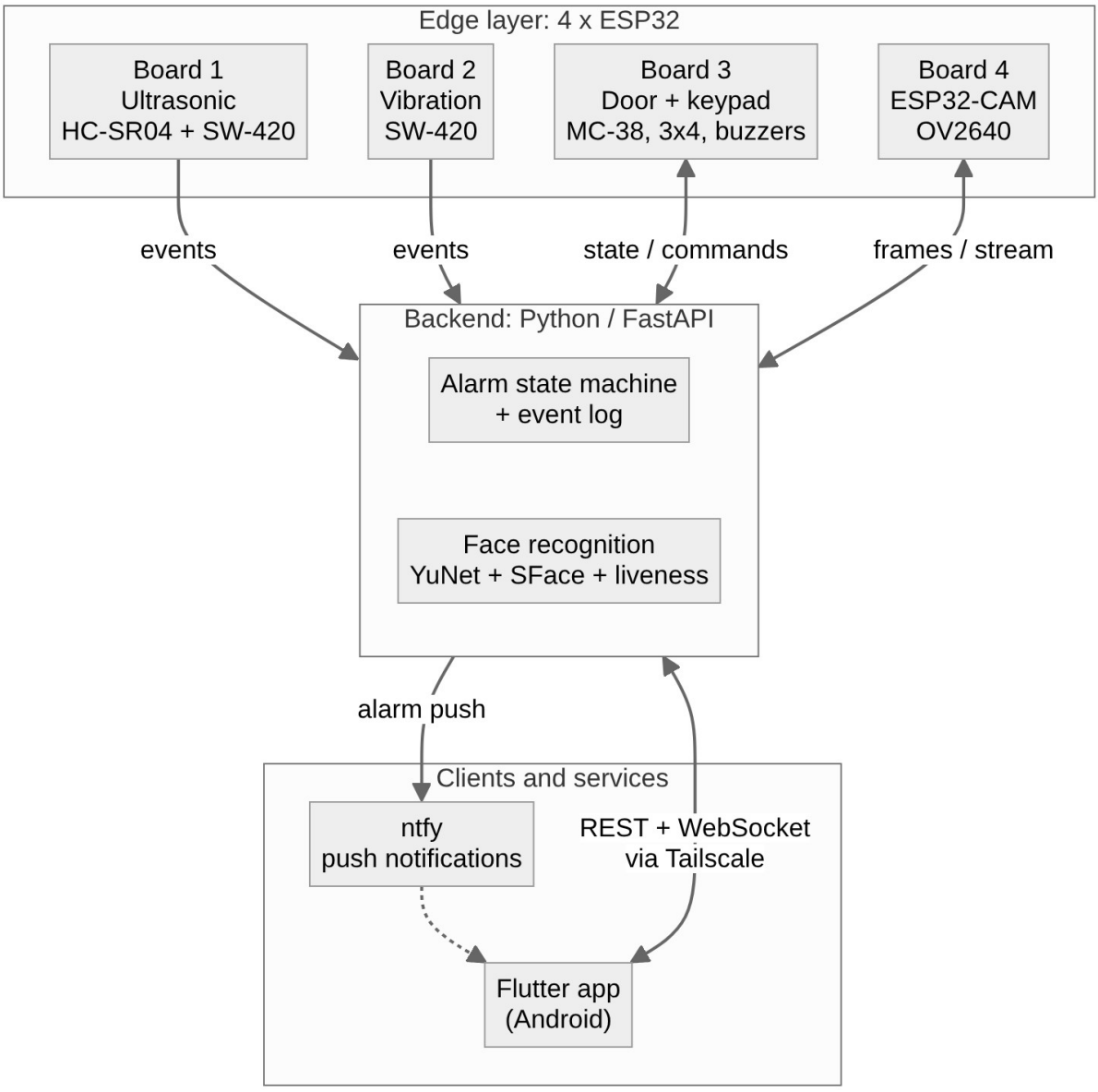
מטרת הפרויקט היא תכנון, מימוש ובדיקה של מערכת אבטחה היברידית מרובת שכבות, המשלבת חיישנים פיזיים, בקרת כניסה, זיהוי פנים וממשק מרחוק. המטרות הוגדרו בשלוש רמות, בהתאם להצעת הפרויקט המקורית:

טבלה 1: מטרות הפרויקט ומצב המימוש

רמה	מטרה	מצב
Must	זיהוי חדירה (חיישן אולטרה-סוני / דלת) או ניסיון פריצה בכוח	הושלם
Must	נטרול המערכת באמצעות קודן פיזי	הושלם
Must	הפעלת אזעקה קולית (Buzzer) כאשר הנטרול אינו מתבצע	הושלם
Nice-to-have	זיהוי פנים ושמירת בנק פרצופים מאושרים	הושלם
Nice-to-have	דאשבורד יומן אירועים בזמן אמת בטלפון	הושלם
Nice-to-have	שינוי הקוד הסודי מהאפליקציה	הושלם
מעבר להצעה	התראות Push באמצעות ntfy גם כשהאפליקציה סגורה	הושלם
מעבר להצעה	דריכה אוטומטית מתוזמנת (Night Mode) וספירה לאחור של Entry Delay	הושלם
מעבר להצעה	זרם וידאו חי (MJPEG) מהמצלמה לאפליקציה	הושלם
לא ממומש	חיוג טלפוני אוטומטי לגורם חירום	לא ממומש — נדרש ספק טלפוניה; במקומו נשלחת התראת הסלמה

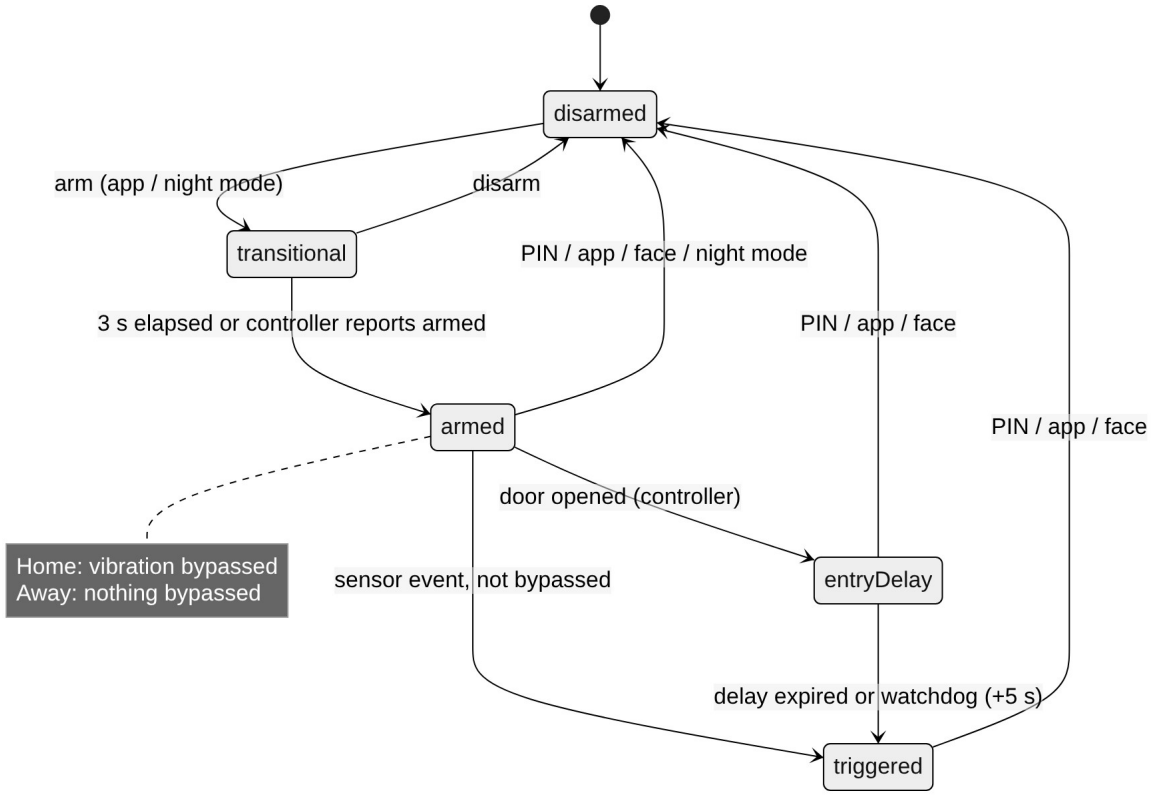
2.4 סקירה כללית של המערכת

המערכת בנויה בארבע שכבות: **(א)** בקרי קצה (ארבעה לוחות ESP32), **(ב)** בקנד מרכזי (FastAPI) המחזיק את מצב האזעקה ומריץ זיהוי פנים, **(ג)** אפליקציית Flutter לאנדרואיד, ו-**(ד)** שירותי תשתית: ntfy להתראות ו-Tailscale לגישה מרחוק. חלוקת האחריות בין הבקרים נקבעה בכוונה לפי עיקרון של "בקרים טיפשים": בקרי החיישנים (אולטרה-סוני ורעידות) מדווחים בלבד ואינם יודעים אם המערכת דרוכה, וההחלטה האם הדיווח מצדיק אזעקה נתונה לבקנד. בקר הדלת, המחזיק את הקודן הפיזי, הוא הסמכות הבלעדית לדריכה ולנטרול, ולפיכך הוא היחיד המקבל פקודות מהבקנד ולא רק שולח.



איור 1: ארכיטקטורת המערכת הכוללת

הקלט למכונת המצבים מגיע משלושה מקורות: חיישנים (אולטרה-סוני, רעידות, דלת מגנטית), הקודן הפיזי והאפליקציה. מצב האזעקה בבקנד עובר בין חמישה מצבים: **Disarmed** (מנוטרל), **Transitional** (חלון יציאה קצר לאחר בקשת דריכה), **Armed** (דרוך, במצב Home או Away), **Entry Delay** (חלון נטרול קצוב לאחר פתיחת דלת) ו-**Triggered** (אזעקה פעילה). בקר הדלת מריץ במקביל מכונת מצבים מקומית משלו, והיא מתוארת בפרק 4. במצב Home מתעלמת המערכת מחיישן הרעידות, כך שניתן לשהות בבית כשהמערכת דרוכה.



איור 2: מכונת המצבים של האזעקה

[הכנס תמונה כאן: צילום של המערכת הפיזית המורכבת — כל ארבעת הלוחות (כולל ה-ESP32-CAM, החיישנים, הקודן והזמזם) מחוברים על שולחן העבודה, עם תיוג מילולי לכל לוח]

איור 3: מימוש פיזי של מערכת HomeGuard

[הכנס תמונה כאן: צילומי מסך של האפליקציה — מסך Dashboard במצב Armed ובמצב Disarmed (תצוגת AlarmStatusIndicator וסטטוס חיישנים), מסך יומן אירועים, ומסך המצלמה]

איור 4: ממשק האפליקציה במצבי המערכת השונים

2.5 התפתחות התכנון לעומת ההצעה המקורית

במהלך הפרויקט נעשו שינויי תכנון משמעותיים לעומת הצעת הפרויקט. השינויים נבעו ממצאי הפיתוח ומשיקולי אמינות ואבטחה, ולא מחוסר יכולת לממש את התכנון המקורי.

טבלה 2: השוואה בין התכנון המקורי למערכת שמומשה

נושא	הצעה מקורית	מערכת שמומשה	נימוק
בקרים	ESP32 מרכזי אחד	ארבעה בקרי ESP32 נפרדים, לוח לכל חיישן	הפרדת תפקידים וכשל מבודד; כל חבר צוות פיתח לוח עצמאי
חיישן חדירה	חיישן ומשדר לייזר (Tripwire)	חיישן אולטרה-סוני HC-SR04	החומרה שהותקנה בפועל היא אולטרה-סוני, והמונחים עודכנו בכל המערכת
מצלמה	מצלמת USB על מחשב	ESP32-CAM עם זרם MJPEG; מצלמת USB נשארה כחלופה	ביטול התלות במחשב צמוד למצלמה; זרם חי לאפליקציה
מכונת מצבים	על גבי ה-ESP32	בבקנד, כשבקר הדלת נשאר הסמכות לדריכה	מצב מרכזי אחד המאחד את כל הדיווחים
ממשק מרחוק	שרת Web על ה-ESP32	אפליקציית Flutter מול בקנד FastAPI (REST + WebSocket)	תמיכה בהתראות, יומן בזמן אמת וסטרימינג
זיהוי פנים	זיהוי פנים בפייתון	YuNet (זיהוי) + SFace (זיהוי זהות) ב-OpenCV	דיוק גבוה מ-Haar+LBPH, ללא תלות בספריות הדורשות קומפילציה נייטיבית
התראות	התראות מהבקר לאפליקציה	ntfy + התראות מקומיות	התראה גם כשהאפליקציה סגורה
תקציב	170 ש"ח	ראה סעיף BOM	—

2.6 חלוקת עבודה

הפרויקט בוצע בשיתוף שני שותפים. שותף א' (גיא בר) אחראי על תת-מערכת הניטור ההיקפי: חיישן אולטרה-סוני, חיישן מגנטי לדלת וממשק השליטה מרחוק. שותף ב' (יאיר בונפיל) אחראי על תת-מערכת ההגנה הפיזית ובקרת הכניסה: חיישן רעידות, קודן 4x4, זמזם, ניהול הסיסמאות וסינון הרעשים. האינטגרציה בין התת-מערכות, הבקנד והאפליקציה בוצעה במשותף.

2.7 מבנה הדוח

הדוח בנוי כך: פרק 3 מציג את הרקע התיאורטי והטכנולוגי ואת סקירת החלופות. פרק 4 מתאר את תכנון המערכת ומימושה: ארכיטקטורה, רכיבי חומרה, קושחה, בקנד ואפליקציה. פרק 5 מציג את הבדיקות, הניסויים והתוצאות, ופרק 6 מסכם, תוך הצגת האתגרים והפתרונות, מגבלות הפרויקט וכיווני המשך. פרק 7 הוא רשימת המקורות, ופרק 8 הנספחים (רשימת חומרים BOM ומבנה הקבצים המצורפים).

⚠️ **תזכורת: [הכנס כאן קישור לווידאו המציג את המערכת עובדת]**

(מומלץ לכלול בסרטון: דריכה מהאפליקציה; חדירה בדלת וספירה לאחור של Entry Delay; נטרול בקודן הפיזי; הפעלת האזעקה הקולית; זיהוי פנים; זרם הווידאו החי; וקבלת התראת ntfy כשהאפליקציה סגורה.)

3. רקע תיאורטי וסקר טכנולוגי

פרק זה סוקר את העקרונות הפיזיקליים, החומרתיים והאלגוריתמיים שעליהם מבוססת המערכת, ואת החלופות הטכנולוגיות שנשקלו.

3.1 מחשוב קצה ומיקרו-בקר ESP32

ה-ESP32 הוא מיקרו-בקר של חברת Espressif הכולל שני ליבות Xtensa LX6 בתדר של עד 240 MHz, 520 KB זיכרון SRAM, ומודול WiFi (802.11 b/g/n, 2.4 GHz) ו-Bluetooth משולבים [1]. ריבוי הליבות והחיבור הרשתי המובנה מאפשרים להריץ במקביל משימות חישה ותקשורת, ולהריץ מערכת הפעלה בזמן אמת (FreeRTOS) תחת סביבת הפיתוח של Arduino. עם זאת, אין בו די משאבים להרצת רשתות נוירונים מודרניות, ולפיכך זיהוי הפנים נעשה בבקנד. מודול ה-ESP32-CAM (מתוצרת AI-Thinker) משלב את ה-ESP32 עם חיישן תמונה OV2640 וזיכרון PSRAM חיצוני, המאפשר אגירת פריימי JPEG [1].

[הכנס תמונה כאן: צילום של לוח ה-ESP32 DevKit V1 ושל מודול ה-ESP32-CAM זה לצד זה, עם סימון הפינים העיקריים בהם נעשה שימוש (או Pinout מתוך מפרט היצרן [1], תוך ציון המקור)]

איור 5: לוחות ה-ESP32 ששימשו בפרויקט

3.2 חיישנים ורכיבי קלט/פלט

3.2.1 חיישן מרחק אולטרה-סוני (HC-SR04). החיישן משדר פולס של גלי קול בתדר 40 kHz לאחר פולס טריגר של 10 µs, ומודד את הזמן שבו האות חוזר מהמכשול (Echo) [2]. המרחק מחושב לפי הנוסחה:

$$d = \frac{v_{sound} \cdot \Delta t}{2}$$

כאשר $v_{sound} \approx 343\,m/s$ (או $0.0343\,cm/\mu s$) בטמפרטורת החדר, Δt הוא זמן ההלוך והשוב, והחלוקה ב-2 נובעת מכך שהגל עובר את הדרך פעמיים [2]. החיישן משמש לזיהוי כניסה או מעבר באזור מוגדר: חציית מרחק הבסיס (Baseline) שנמדד בהתקנה מזוהה כחדירה.

3.2.2 חיישן רעידות (SW-420). המודול כולל חיישן רעידות מכני ומשווה מתח (LM393 [3]) הממיר את הרעידה לאות דיגיטלי. סף הרגישות נקבע באמצעות פוטנציומטר. החיישן מזהה ניסיון פריצה בכוח (מכה בדלת או בחלון), והעיבוד כולל סינון רעשים והגדרת סף "כוח מוגזם".

3.2.3 חיישן מגנטי לדלת (MC-38). חיישן מסוג Reed Switch, המורכב ממגען שנסגר בנוכחות שדה מגנטי. כאשר הדלת נפתחת, המגנט מתרחק והמגען נפתח, והבקר קורא שינוי במצב הפין הדיגיטלי.

3.2.4 קודן מטריצה 3x4. הקודן בנוי מטריצה של ארבע שורות ושלוש עמודות (ספרות 0–9 והמקשים * ו-#). הבקר מזין שורה אחת בכל פעם ומאתר איזו עמודה נסגרת, ובכך מזהה את הלחצן שנלחץ באמצעות שבעה פינים בלבד [4]. (הצעת הפרויקט המקורית ציינה קודן 4x4; הקודן שהותקן בפועל הוא 3x4.)

3.2.5 זמזם (Buzzer). רכיב פלט המפיק צליל להתראה קולית, ומשמש גם למתן משוב לקודן.

[הכנס תמונה כאן: צילום או סכמת חיבורים (Fritzing / KiCad) של שלושת בקרי החיישנים — פינים, נגדים ומקורות מתח — עם תיוג הפינים בפועל מתוך הקוד]

איור 6: סכמת החיבורים של רכיבי החישה והקלט/פלט

3.3 מכונת מצבים (Finite State Machine)

מכונת מצבים היא מודל חישובי שבו המערכת נמצאת בכל רגע באחד ממספר סופי של מצבים, והמעבר ביניהם מתרחש בהתאם לאירועי קלט. מודל זה מתאים במיוחד למערכות אזעקה, שבהן ההתנהגות הרצויה לאירוע זהה (למשל פתיחת דלת) תלויה במצב הנוכחי (דרוך לעומת מנוטרל). יתרון נוסף הוא קביעות: שליטה מפורשת בטבלת המעברים מונעת מצבים סותרים, כגון אזעקה פעילה במערכת שהוגדרה כמנוטרלת.

3.4 ארכיטקטורת תקשורת

3.4.1 HTTP ו-REST. הבקרים מדווחים לבקנד באמצעות בקשות HTTP POST, ופקודות מהאפליקציה מועברות במודל REST — מערכת משאבים הנגישה באמצעות פעלים סטנדרטיים של HTTP [5].

3.4.2 WebSocket. פרוטוקול WebSocket מאפשר ערוץ דו-כיווני מתמשך מעל חיבור TCP יחיד [6]. הוא משמש לדחיפת שינויי מצב ואירועים לאפליקציה בזמן אמת, ללא צורך בדגימה תקופתית (Polling).

3.4.3 זרם וידאו MJPEG. MJPEG (Motion JPEG) הוא זרם של תמונות JPEG עוקבות המועברות כ-`multipart/x-mixed-replace`. הוא פשוט למימוש על בקרים מוגבלי משאבים, אך אינו נתמך באופן מובנה ב-Flutter, ולפיכך הפענוח בצד האפליקציה מומש ידנית. בצד הבקר נעשה

שימוש במודול `esp_http_server` של ESP-IDF, המריץ את מטפל הבקשות ב-Task נפרד [7] ובכך אינו חוסם את לולאת הקוד הראשית, שבה מתבצעת שליחת הפריימים לזיהוי פנים.

3.4.4 בקנד FastAPI. FastAPI הוא מסגרת (Framework) אסינכרונית ל-Python המבוססת על תקן ASGI ועל מנגנון הולידציה Pydantic [8]. היא נבחרה בשל תמיכה מובנית ב-WebSocket, תיעוד API אוטומטי (OpenAPI) ובשל המודל האסינכרוני המאפשר לשרת אחד לטפל בו-זמנית בדיווחי לוחות, בחיבורי אפליקציה ובזרם הווידאו.

3.5 ראייה ממוחשבת וזיהוי פנים

זיהוי פנים כולל שני שלבים: **זיהוי** (Detection) — איתור מיקום הפנים בתמונה, ו**זיהוי זהות** (Recognition) — השוואת הפנים שנמצאו לבנק פנים מאושרים.

3.5.1 גישה קלאסית: Haar Cascade ו-LBPH. האלגוריתם של Viola-Jones משתמש במסווג מדורג (Cascade) המבוסס על מאפייני Haar, ומאפשר זיהוי פנים מהיר במעבד רגיל [9]. הוא רגיש לתאורה ולזווית צילום ופחות מדויק בזיהוי זהות. גישה זו שימשה כבסיס ראשוני, ונשארה כגיבוי במקרה שקבצי המודלים העדכניים חסרים.

3.5.2 גישה מודרנית: YuNet ו-SFace. YuNet הוא גלאי פנים קל על בסיס רשת קונבולוציה, המיועד לריצה בזמן אמת על מעבד [10]. SFace הוא מודל זיהוי זהות המפיק לכל פנים וקטור מאפיינים (Embedding) והוא מאומן באמצעות פונקציית הפסד Sigmoid-Constrained Hypersphere Loss [11]. ההשוואה בין שני פנים נעשית באמצעות דמיון קוסינוס בין הווקטורים:

$$\cos(\theta) = \frac{a \cdot b}{\|a\| \|b\|}$$

כאשר הערך מושווה לסף שנקבע. שני המודלים נטענים באמצעות מודול ה-DNN של OpenCV [12], ללא תלות בספריות הדורשות קומפילציה נייטיבית.

3.5.3 זיהוי חיות (Liveness / Anti-Spoofing). SFace משווה זהות בלבד ואינו מבחין בין אדם אמיתי לתמונה מודפסת או למסך. לפיכך נוספו בבקנד שתי בדיקות חיות בלתי תלויות, ששתיהן חייבות להיכשר כדי שזיהוי פנים יורשה לנטרל את האזעקה: **(א)** בדיקת תנועה — מעקב אחר חמש נקודות הציון (עיניים, אף, זוויות פה) ש-YuNet מחזיר, ובדיקה שבחלון של 2.5 שניות קיימת תזוזה זעירה טבעית; **(ב)** מודל MiniFASNetV2 [18], המסווג חתך תמונה של הפנים ושל סביבתן כאמיתי או כזיוף (הדפסה / הקרנה). בהיעדר קובץ המודל המערכת נכשלת לכיוון הבטוח, כלומר לא מנטרלת. עם זאת, ספי ההחלטה של שתי הבדיקות הם הערכות ראשוניות ולא נמדדו מול מערך

ניסיונות הונאה מסודר, ובדיקת התנועה אינה מגנה מפני הקרנת וידאו. בשל כך נטרול לפי פנים מוגדר כאופציה שכבויה כברירת מחדל, וככלי נוחות ולא כמנגנון אבטחה.

[הכנס תמונה כאן: תמונה לדוגמה מהמצלמה עם תיבת זיהוי סביב פנים מוכרות (עם שם) ותיבה נוספת סביב פנים לא מוכרות, כפי שמוצג באפליקציה (מסך Camera/Faces)]

איור 7: דוגמה לזיהוי פנים במערכת

3.6 אבטחת מידע ותקשורת מאובטחת

מערכת אזעקה היא יעד תקיפה אטרקטיבי, ולכן אבטחת המידע נשקלה כחלק מהתכנון ולא כתוספת:

- **סוגי אישורים:** האפליקציה מקבלת אסימון סשן אקראי (בתוקף 12 שעות) לאחר הזדהות בסיסמה, והלוחות מזדהים באמצעות מפתח מכשיר משותף. הפרדה זו נובעת מכך שלוח אינו יכול לבצע הזדהות אינטראקטיבית.
- **שמירת סיסמה:** הסיסמה נשמרת כ-hash באמצעות scrypt עם Salt, פונקציית גזירת מפתח הדורשת זיכרון רב ומעכבת תקיפות ניחוש [13]. ההשוואה נעשית בזמן קבוע (`compare_digest`) למניעת תקיפות תזמון.
- **נעילה:** לאחר שלושה ניסיונות שגויים ננעלת הכניסה למשך 30 שניות. הבקנד הוא הסמכות היחידה לנעילה, שכן נעילה בצד האפליקציה ניתנת לעקיפה על ידי הפעלה מחדש.
- **גישה מרחוק באמצעות Tailscale:** Tailscale הוא רשת פרטית (VPN) המבוססת על פרוטוקול WireGuard [14][15]. המחשב והטלפון יוזמים חיבור יוצא בלבד, ולפיכך אין צורך לפתוח אף פורט לרשת האינטרנט הפתוחה. עיקרון זה — המכשיר הביתי יוזם את החיבור ואינו מאזין לנכנס — משמש גם מערכות מסחריות, ונבחר כאן מפני שאימות המערכת מבוסס סיסמה משותפת יחידה ואינו ראוי לחשיפה לרשת הפתוחה.

3.7 התראות

התראות מקומיות באפליקציה (`flutter_local_notifications`) פועלות רק כל עוד האפליקציה חיה ברקע. כדי לספק התראה גם כשהאפליקציה סגורה נשקלה הצורה הסטנדרטית באנדרואיד — Firebase Cloud Messaging (FCM) — אך היא מחייבת פרויקט Firebase וקובץ הגדרות ייעודי (`google-services.json`), ובהיעדרו בניית האפליקציה נכשלת. לפיכך נבחר שירות ntfy [16], המבוסס על פרסום והרשמה (Publish/Subscribe) באמצעות HTTP ואינו מחייב חשבון. שם ה-Topic משמש כסוד, ולפיכך אינו נרשם ביומן האירועים.

3.8 אפליקציית Flutter

Flutter היא מסגרת של Google לפיתוח ממשקי משתמש בשפת Dart [17]. האפליקציה מפותחת לאנדרואיד בלבד, ומנהלת מצב באמצעות Provider, כאשר מצב האזעקה והאימות נטענים לפני

בניית הממשק ומאזינים לערוץ ה-WebSocket. העיצוב הוא Material 3 עם פלטת צבעים מכוונת: אדום מציין מצב דרוך או אזעקה, וירוק מציין מצב מנוטרל.

3.9 סביבות וכלי פיתוח

טבלה 3: כלי הפיתוח

תחום	כלי / סביבה
קושחה	Arduino IDE / arduino-cli, ליבת esp32:esp32 (C++)
בקנד	Python, FastAPI, Uvicorn, OpenCV (opencv-contrib-python), Pytest, Ruff
אפליקציה	Flutter / Dart, Android SDK, JDK 17 (בנייה דרך WSL2)
בקרת גרסאות ו-CI	Git, GitHub Actions
תרשימים	*(להשלים: כלי ששימש לשרטוט דיאגרמות הבלוקים ותרשימי הזרימה)*
חיבורי חומרה	*(להשלים: KiCad / Fritzing / אחר)*

(הרשימה המלאה והמאוחדת של כל המקורות מופיעה בפרק 7 בקובץ `_03_tests_summary_bom_refs.md`.)

4. תכנון ומימוש המערכת

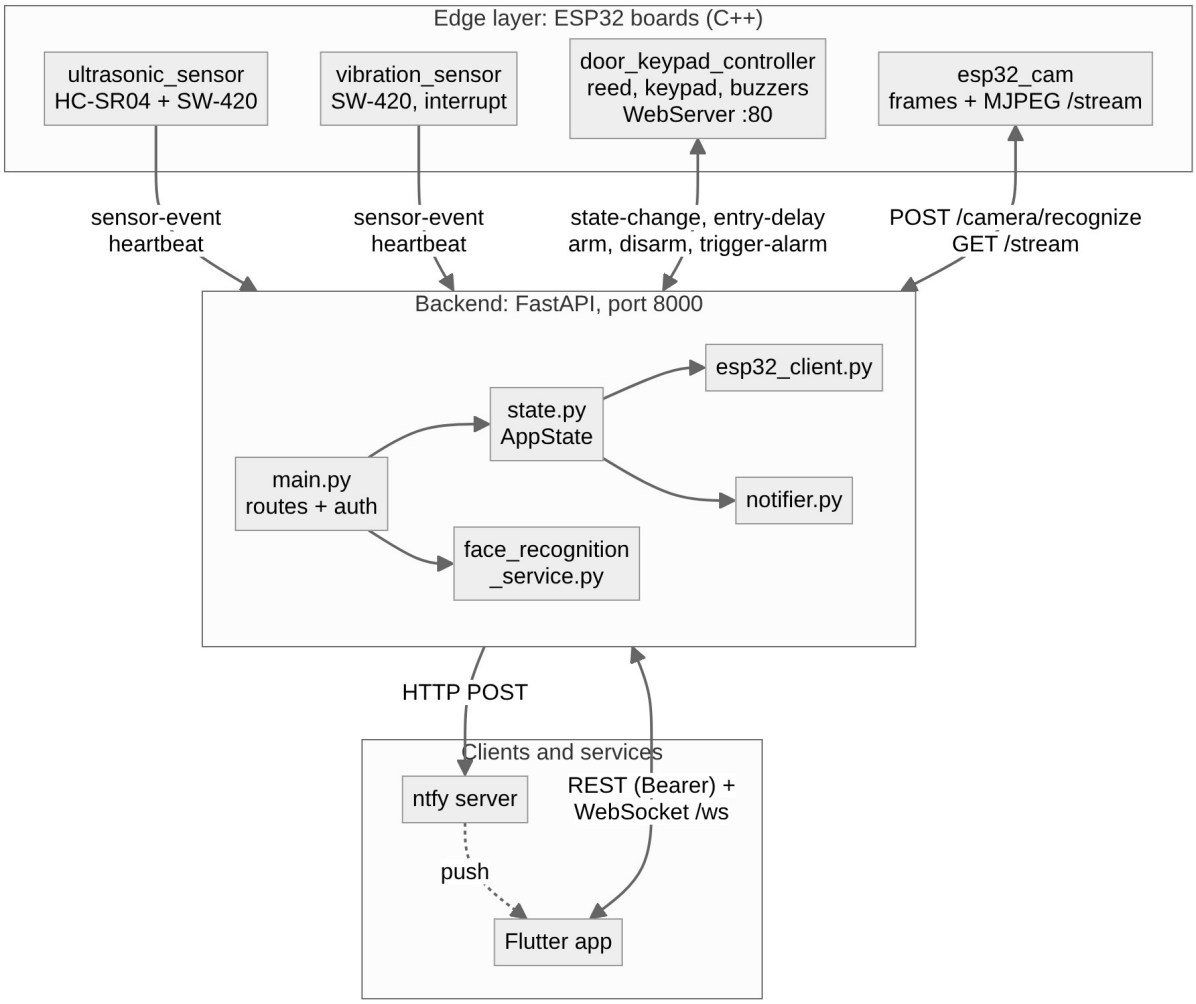
פרק זה מתאר את תכנון המערכת ואת מימושה, מהארכיטקטורה הכוללת ועד לרמת הפונקציות המרכזיות בקושחה, בבקנד ובאפליקציה. הקוד המלא מצורף בנספחים; בגוף הפרק מוצגים הסברים וקטעים נבחרים בלבד.

4.1 ארכיטקטורה כללית וזרימת נתונים

4.1.1 תיאור תרשים הבלוקים

המערכת בנויה כארבע שכבות הפועלות יחד (איור 8):

1. **שכבת הקצה (Edge):** ארבעה בקרי ESP32 עצמאיים, שכל אחד מהם מחובר לרשת WiFi ופועל ללא תלות בשאר. שלושה מהם (אולטרה-סוני, רעידות ובקר הדלת) מדווחים לבקנד בבקשות HTTP POST, ורביעי (ESP32-CAM) דוחף פריימים לזיהוי פנים ומגיש זרם וידאו.
2. **שכבת הבקנד:** שרת FastAPI יחיד המחזיק את מצב האזעקה, מאמת כל פנייה, מריץ את זיהוי הפנים, שולח התראות ומשרת את האפליקציה.
3. **שכבת האפליקציה:** לקוח Flutter לאנדרואיד, המתקשר עם הבקנד באמצעות REST (פקודות) ו-WebSocket (דחיפת מצב בזמן אמת).
4. **שכבת התשתית:** שירות ntfy להתראות Push וביטול הצורך בחשיפת הבקנד לאינטרנט הפתוח באמצעות רשת פרטית (Tailscale).



איור 8: תרשים בלוקים מפורט של מערכת HomeGuard

4.1.2 עקרונות התכנון

חמשת העקרונות שהנחו את התכנון:

- **בקרים "טיפשים" בחיישנים:** לוחות האולטרה-סוני והרעידות אינם יודעים אם המערכת דרוכה. הם מדווחים על כל אירוע, והבקנד מחליט אם הוא מצדיק אזעקה. כך אין צורך לעדכן קושחה בעת שינוי מדיניות (למשל מצב Home).
- **סמכות יחידה לדריכה ולנטרול:** בקר הדלת, המחזיק את הקודן הפיזי, הוא הסמכות הסופית לשאלה "האם המערכת דרוכה". הבקנד והאפליקציה משקפים את מצבו, ופקודות מהאפליקציה מועברות אליו.
- **שקיפות כשל:** מצב שאינו ידוע אינו מוצג כתקין. כל אחד משלושת בקרי החיישנים והדלת שולח אות חיים (Heartbeat) כל 10 שניות (ללוח המצלמה אין Heartbeat נפרד, כי הפריימים עצמם משמשים אות חיים), וכל פקודה מהאפליקציה מחזירה את הדגל `hardware`, המציין אם הלוח אישר אותה בפועל.
- **נתיב ההתראה אינו חוסם:** שליחת התראה חיצונית, וכן דיווחי הבקרים, אינם עוצרים את עדכון האפליקציות בזמן אמת (ראה סעיפים 4.3.4 ו-4.4.6).
- **הגנה בשכבות:** שני סוגי אישורים, הפרדה בין סיסמת אפליקציה למפתח מכשיר, ורשת פרטית לגישה מרחוק (סעיף 4.6).

4.1.3 ממשקי התקשורת (API)

כל ה-endpoints מוגנים, למעט `/login`, `health` ותיעוד ה-API האוטומטי. הטבלה מסכמת את הממשקים לפי הצד הקורא:

טבלה 4: ממשקי התקשורת של המערכת

כיוון	נתיב	אימות	תפקיד
לוח → בקנד	`POST /esp32/sensor-event?sensor=&board=&reason=`	X-Device-Key	דיווח חדירה (אולטרה-סוני / רעידות / דלת)
לוח → בקנד	`POST /esp32/heartbeat?board=[&fault=]`	X-Device-Key	אות חיים, ובמקרה הצורך קוד תקלה
בקר דלת → בקנד	`POST /esp32/state-change ({"armed":bool})`	X-Device-Key	מצב דריכה סמכותי
בקר דלת → בקנד	`POST /esp32/entry-delay ({"seconds":n})`	X-Device-Key	תחילת ספירת נטרול לאחר פתיחת דלת
ESP32-CAM → בקנד	`POST /camera/recognize` (multipart, שדה `photo`)	X-Device-Key	פריים לזיהוי פנים
בקנד → בקר דלת	`POST /arm`, `/disarm`, `/trigger-alarm`, `/keypad-pin`	X-Device-Key	פקודות שליטה, זמזום וחילוף קוד
בקנד → ESP32-CAM	`GET /stream`	X-Device-Key	משיכת זרם MJPEG
אפליקציה → בקנד	`POST /login`, `/arm`, `/disarm`, `/change-password`, `/faces`, `//settings*` ועוד	Bearer Token	שליטה והגדרות
אפליקציה → בקנד	`GET /state`, `GET /camera/stream`	Bearer Token	מצב נוכחי וסטרימינג וידאו
אפליקציה ↔ בקנד	`WebSocket /ws?token=`	Token בשאילתה	דחיפת מצב בזמן אמת

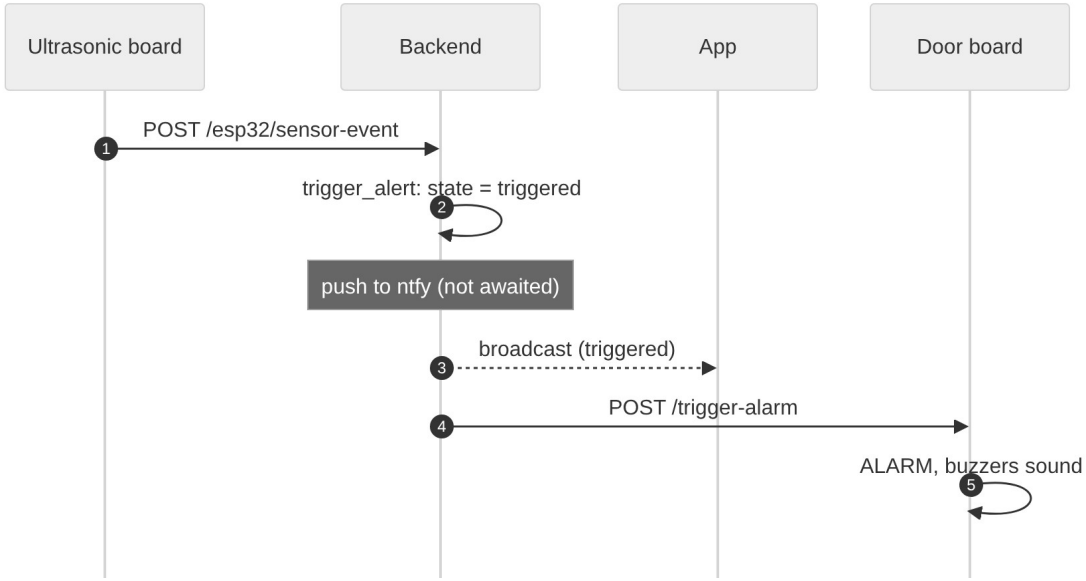
4.1.4 תרחישי זרימה מרכזיים

תרחיש 1 — חדירה דרך חיישן היקפי (מערכת דרוכה). (1) לוח האולטרה-סוני מזהה חמש קריאות רצופות הקרובות מהבסיס, ושולח (2) `POST /esp32/sensor-event`. הבקנד מאמת את מפתח המכשיר ומסמן את הלוח כחי. (3) `trigger_alert` בודק שהמערכת דרוכה ושהחיישן אינו מעקוף, ועובר למצב Triggered. (4) נרשמת שורה ביומן, נשלחת התראת ntfy (ללא המתנה לתשובה), ומתחיל טיימר הסלמה. (5) המצב נדחף בשידור WebSocket לכל האפליקציות. (6) רק לאחר מכן נשלחת לבקר הדלת הפקודה `/trigger-alarm`, והזמזמים מופעלים.

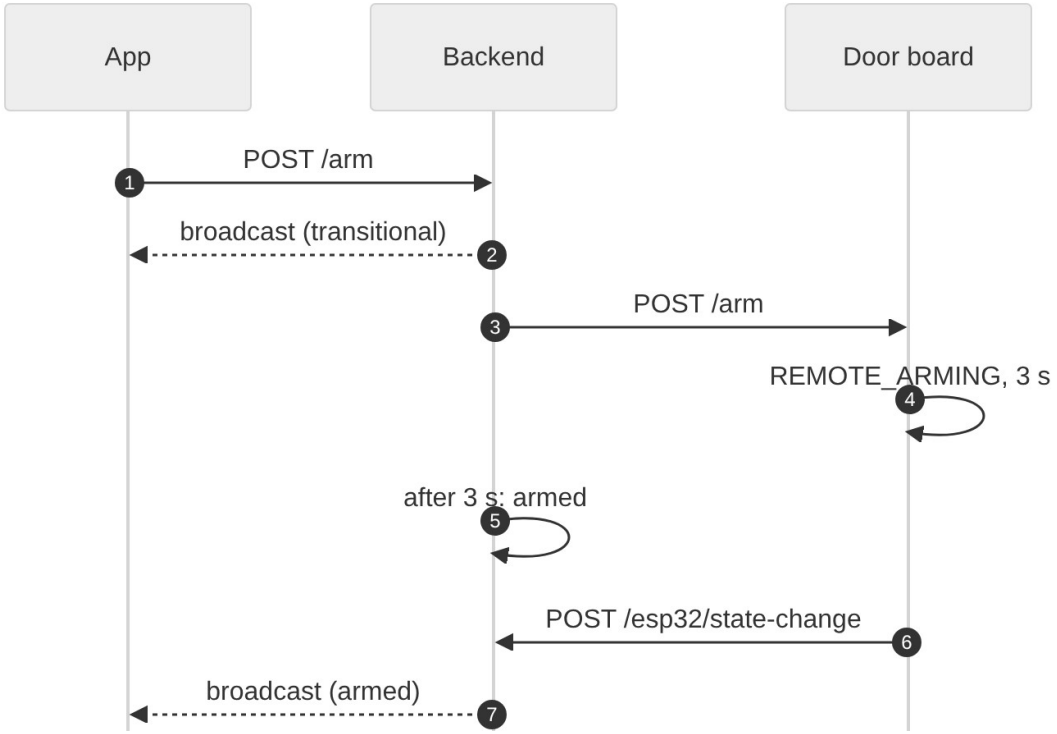
תרחיש 2 — דריכה מהאפליקציה. (1) הבקנד עובר מיד ל-Transitional ומשדר `POST /arm`, כך שהאפליקציה מציגה ספירה לאחור. (2) במקביל נשלחת פקודה לבקר הדלת, שעובר ל-REMOTE_ARMING. (3) לאחר שלוש שניות שני הצדדים עוברים ל-Armed, ובקר הדלת מדווח `state-change` כדי לאשר את המצב.

תרחיש 3 — כניסה לגיטימית דרך הדלת. (1) הדלת נפתחת על מערכת דרוכה, ובקר הדלת עובר ל-ENTRY_DELAY ומדווח (2) `entry-delay`. האפליקציה מציגה ספירה לאחור של 25 שניות. (3) הקשת קוד נכון ו-# מנטרלת ומדווחת `state-change`; אחרת, בתום הזמן עובר הבקר ל-ALARM ומדווח חדירה. (4) אם הבקר שותק, הבקנד מעלה אזעקה בעצמו לאחר זמן הנטרול ועוד חמש שניות (ראה 4.4.2).

Scenario 1: intrusion through a perimeter sensor (system armed)



Scenario 2: arming from the app



Scenario 3: legitimate entry through the door



איור 9: תרשימי רצף של שלושת תרחישי השימוש המרכזיים

4.2 מימוש החומרה

4.2.1 הלוחות, הרכיבים והפינים

טבלה 5: חלוקת הרכיבים והפינים בין הלוחות

לוח	רכיבים	פינים (GPIO)	תפקיד
1 — אולטרה-סוני	HC-SR04; חיישן כוח SW-420 נוסף	TRIG=14, ECHO=12; חיישן כוח=27	ניטור היקפי + זיהוי כוח בחלון סמוך
2 — רעידות	חיישן רעידות SW-420	יציאה דיגיטלית=13	זיהוי פריצה בכוח
3 — דלת וקודן	חיישן מגנטי MC-38 (Reed); קודן 3x4; שני זמזמים	דלת=4 (INPUT_PULLUP); זמזמים=23, 22; שורות=25, 13, 12, 27; עמודות=26, 33, 14	סמכות דריכה/נטרול, אזעקה קולית

לוח	רכיבים	פינים (GPIO)	תפקיד
4 — מצלמה	ESP32-CAM (AI-Thinker, חיישן OV2640)	פינוט הלוח הקבוע של AI-Thinker (למשל XCLK=0, SIOD=26, SIOC=27, PCLK=22)	זיהוי פנים וזרם וידאו חי

על לוח 1 מותקן חיישן הכוח השני (רכיב זהה ללוח 2), מפני שהוא ממוקם פיזית ליד החיישן האולטרה-סוני, ולכן לא נדרש לוח נוסף רק בשבילו. הוא מדווח כ-`sensor=vibration` עם `board=ultrasonic`.

4.2.2 פירוט הלוחות

לוח אולטרה-סוני. החיישן HC-SR04 מחובר לפיני TRIG (פלט) ו-ECHO (קלט). הפולס שנמדד על ECHO נקרא באמצעות `pulseIn`, והמרחק מחושב לפי הנוסחה בסעיף 3.2.1. חיישן הכוח מחובר לפין 27 ומחובר בפסיקה (ראה 4.3.3).

[הכנס תמונה כאן: שרטוט חשמלי של לוח האולטרה-סוני — ESP32 DevKit V1, HC-SR04 (VCC, GND, TRIG→GPIO14, ECHO→GPIO12) עם מחלק מתח אם מומש), וחיישן SW-420 (OUT→GPIO27). כולל סימון הספק (5V/3.3V)]

איור 10: סכמת חיבורים — לוח אולטרה-סוני

לוח רעידות. חיישן SW-420 בעל משווה LM393; הפלט הדיגיטלי מחובר לפין 13. סף הרגישות נקבע בפוטנציומטר שעל המודול.

[הכנס תמונה כאן: שרטוט חשמלי / צילום של לוח הרעידות — ESP32 + SW-420 (VCC, GND, OUT→GPIO13), עם סימון הפוטנציומטר]

איור 11: סכמת חיבורים — לוח הרעידות

בקר הדלת והקודן. הוא הלוח המורכב ביותר: חיישן מגנטי על פין 4 עם נגד Pull-Up פנימי (דלת סגורה = LOW, פתוחה = HIGH), שני זמזמים על פינים 23 ו-22, וקודן מטריצה 3x4. שורות הקודן מוגדרות כפלט (במצב סרק HIGH), והעמודות כקלט עם Pull-Up פנימי, כך שאין צורך בנגדים חיצוניים.

[הכנס תמונה כאן: שרטוט חשמלי של בקר הדלת — ESP32, חיישן Reed (GPIO4) ל-GND), שני זמזמים (GPIO23, GPIO22), ומטריצת הקודן עם שורות GPIO25, 13, 12, 27 → R1–R4 ועמודות GPIO26, 33, 14 → C1–C3]

איור 12: סכמת חיבורים — בקר הדלת והקודן

ESP32-CAM. מודול AI-Thinker כולל את חיישן ה-OV2640 ואת זיכרון ה-PSRAM. אין בו מעגל איפוס אוטומטי, ולכן כל צריבה מחייבת רצף ידני (החזקת BOOT, לחיצה על RESET ושחרורה, שחרור BOOT והתחלת ההעלאה), ואחריה לחיצה נוספת על RESET להרצת הקוד החדש.

[הכנס תמונה כאן: צילום של ה-ESP32-CAM מחובר לממיר USB-Serial בזמן צריבה, או שרטוט החיבור הזמני (IO0 ל-GND בעת צריבה)]

איור 13: חיבור ה-ESP32-CAM לצריבה

4.2.3 מארזים, הדפסה תלת-ממדית והרכבה

[הכנס תמונה כאן: מודל תלת-ממדי (3D) של מארזי הלוחות / מעמד המצלמה / תושבת החיישן האולטרה-סוני — צילום מסך מתוכנת המידול, ולצידו תמונה של החלק המודפס. אם לא נעשה שימוש במארז מודפס — להחליף בצילום ההרכבה בפועל ולמחוק את הסעיף]

איור 14: מודל תלת-ממדי וחלקים מודפסים

[הכנס תמונה כאן: צילום של המערכת הפיזית המורכבת בבית/על השולחן, עם סימון מיקום כל לוח וחיישן (היקף, דלת, חלון, מצלמה)]

איור 15: הרכבת המערכת בפועל

רשימת החומרה המלאה (BOM) והעלויות מופיעה בנספח א'.

4.3 מימוש הקושחה (C++, ארבעת בקרי ה-ESP32)

4.3.1 מבנה משותף לכל הלוחות

ארבעת הקבצים (`.ino`) חולקים את שלוש ההחלטות הראשונות הבאות, ושתי האחרונות (Cooldown ו-Heartbeat) רלוונטיות ללוחות החיישנים והדלת:

- **הפרדת סודות מהקוד:** פרטי ה-WiFi, כתובת הבקנד, מפתח המכשיר וקוד הקודן הברירת-מחדל נשמרים בקובץ `secrets.h` נפרד, שאינו נכלל במאגר הקוד (מצורפת תבנית בלבד).
- **אימות בכל דיווח:** כל בקשה מכילה את הכותרת `X-Device-Key`. לוח שמפתחו אינו תואם מקבל 401 ומדפיס זאת בטרמינל.
- **הגבלת זמן בקשה:** ברירת המחדל של ליבת Arduino היא 5 שניות לחיבור ו-5 שניות לתגובה, כלומר עד 10 שניות חסימה. הוגדרה מגבלה של שנייה אחת לכל שלב (`HTTP_TIMEOUT_MS`), כך שבקשה בודדת לא מעוורת את הלוח למשך זמן שבו ניתן לעבור בחיישן. הדיווחים מבוצעים ללא ניסיון חוזר, ולפיכך המחיר של המגבלה הוא דיווח שאבד ולא דיווח שגוי.
- **הגבלת קצב (Cooldown) בלוחות החיישנים:** לאחר דיווח, אירועים נוספים של אותו חיישן אינם מדווחים במשך 5 שניות, אך הדגימה נמשכת. כך מוגבל עומס הרשת מבלי להסתיר אירוע מתמשך.
- **אות חיים:** בלוחות החיישנים ובבקר הדלת נשלח כל 10 שניות `POST /esp32/heartbeat?board=<שם>`. הבקנד מסמן לוח כלא מחובר לאחר 30 שניות שקט.

4.3.2 לוח האולטרה-סוני

מדידת מרחק — `getDistance()`. הפונקציה שולחת פולס טריגר של 10 µs ומודדת את משך פולס ה-ECHO באמצעות `pulseIn` עם מגבלת זמן של 30 ms. מגבלה זו מונעת תקיעה כאשר אין הד חוזר, ומגדירה טווח מרבי של בערך $30000/2=510 cm$,$0.034 \cdot$, מעבר לטווח המתוכנן של החיישן (400 ס"מ [2]). החזרת 0 מציינת "אין הד".

```
207 float getDistance() {
208   digitalWrite(trigPin, LOW);
209   delayMicroseconds(2);
210   digitalWrite(trigPin, HIGH);
211   delayMicroseconds(10);
212   digitalWrite(trigPin, LOW);
213
214   // ה-timeout (30000 30 = שניות-מיקרוms) מונע מהקוד להיתקע אם אין הד
215   // (למשל אם אין שום דבר בטווח האפקטיבי של החיישן) חוזר.
216   long duration = pulseIn(echoPin, HIGH, 30000);
217
218   if (duration == 0) return 0; // חריגה מטווח או שגיאת קריאה
219
220   // מהירות הקול היא בערך 0.034 ס"מ למיקרו-שנייה. מחלקים ב-2 כי
221   // (מסלול כפול) המדידה כוללת את הזמן שלוקח לקול ללכת ולחזור.
222   return duration * 0.034 / 2;
223 }
```

קוד 1: מדידת מרחק אולטרה-סונית

כיול אוטומטי של מרחק הבסיס — `calibrateBaseline()`. בכל הפעלה נדגמות 10 קריאות תקינות והממוצע שלהן הופך למרחק הבסיס (המרחק לקיר או למסדרון הנצפה). כך החיישן

מתאים את עצמו למיקום ההתקנה בפועל, ללא קיבוע ערך בקוד. כדי שכיול שגוי לא ישבית את החיישן, נוספו בדיקות שפיות:

- בסיס שנמדד חייב להימצא בטווח 100–400 ס"מ. ערך קצר יותר מצביע על מכשול שחסם את הקרן בזמן הכיול (למשל אדם שעמד מול החיישן בעת הפסקת חשמל), ויקבע בסיס שממנו שום מסדרון ריק לא ייחשב חריגה.
- ערך שנדחה אינו מאומץ; הלוח חוזר לבסיס האחרון שנשמר בזיכרון הקבוע ([19] NVS) או לברירת המחדל (200 ס"מ), ומדווח את התקלה `baseline_rejected`.
- קפיצה של יותר מ-40 ס"מ מהבסיס השמור מדווחת כ-`baseline_moved`, אך אינה נדחית (סביר שהחיישן הוזז או כוון מחדש).
- התקלה נשלחת על גבי ה-Heartbeat ולא על `sensor-event`, כדי שתקלת כיול לא תפעיל אזעקה מלאה במערכת דרוכה.

```
230 void calibrateBaseline() {
231   Serial.println("Calibrating baseline distance...");
232   float sum = 0;
233   int collected = 0;
234   int attempts = 0;
235   const int MAX_ATTEMPTS = 50; // תקרת ביטחון - לא נתקעים לנצח אם אין הד בכלל
236 
237   while (collected < CALIBRATION_SAMPLES && attempts < MAX_ATTEMPTS) {
238     float d = getDistance();
239     attempts++;
240     if (d > 0) {
241       collected++;
242       sum += d;
243       Serial.printf("  sample %d/%d: %.1f cm\n", collected, CALIBRATION_SAMPLES, d);
244     }
245     delay(60); // מרווח קצר בין דגימות, שהד קודם ידעך לפני הבאה
246   }
247 
248   if (collected == 0) {
249     // ...
253     Serial.printf("WARNING: calibration got no valid reading — keeping %.1f cm.\n",
    DEFAULT_DISTANCE);
254     setFault(FAULT_BASELINE);
255     return;
256   }
257 
258   float measured = sum / collected;
259 
260   if (!isPlausibleBaseline(measured)) {
261     // הערך נמדד, הוא פשוט לא יכול להיות התקנה אמיתית (ראה BASELINE_MIN_CM).
262     // נופלים חזרה למה שכבר ידוע — הבסיס השמור, או ברירת המחדל — ומדווחים.
263     Serial.printf("WARNING: measured baseline %.1f cm is outside %.0f-%.0f cm —
    rejected, keeping %.1f cm.\n",
264                   measured, BASELINE_MIN_CM, BASELINE_MAX_CM, DEFAULT_DISTANCE);
265     setFault(FAULT_BASELINE);
266     return;
267   }
268 
269   // ...
272   if (haveStoredBaseline && fabs(measured - DEFAULT_DISTANCE) > BASELINE_DRIFT_CM) {
273     Serial.printf("WARNING: baseline moved %.1f cm -> %.1f cm since the last boot.\n",
274                   DEFAULT_DISTANCE, measured);
275     setFault(FAULT_BASELINE_MOVED);
276   }
277 
278   DEFAULT_DISTANCE = measured;
279   storeBaseline(measured);
280   Serial.printf("Baseline calibrated: %.1f cm (from %d readings)\n", DEFAULT_DISTANCE,
    collected);
281 }
282 
```

קוד 2: כיול הבסיס ובדיקות השפיות

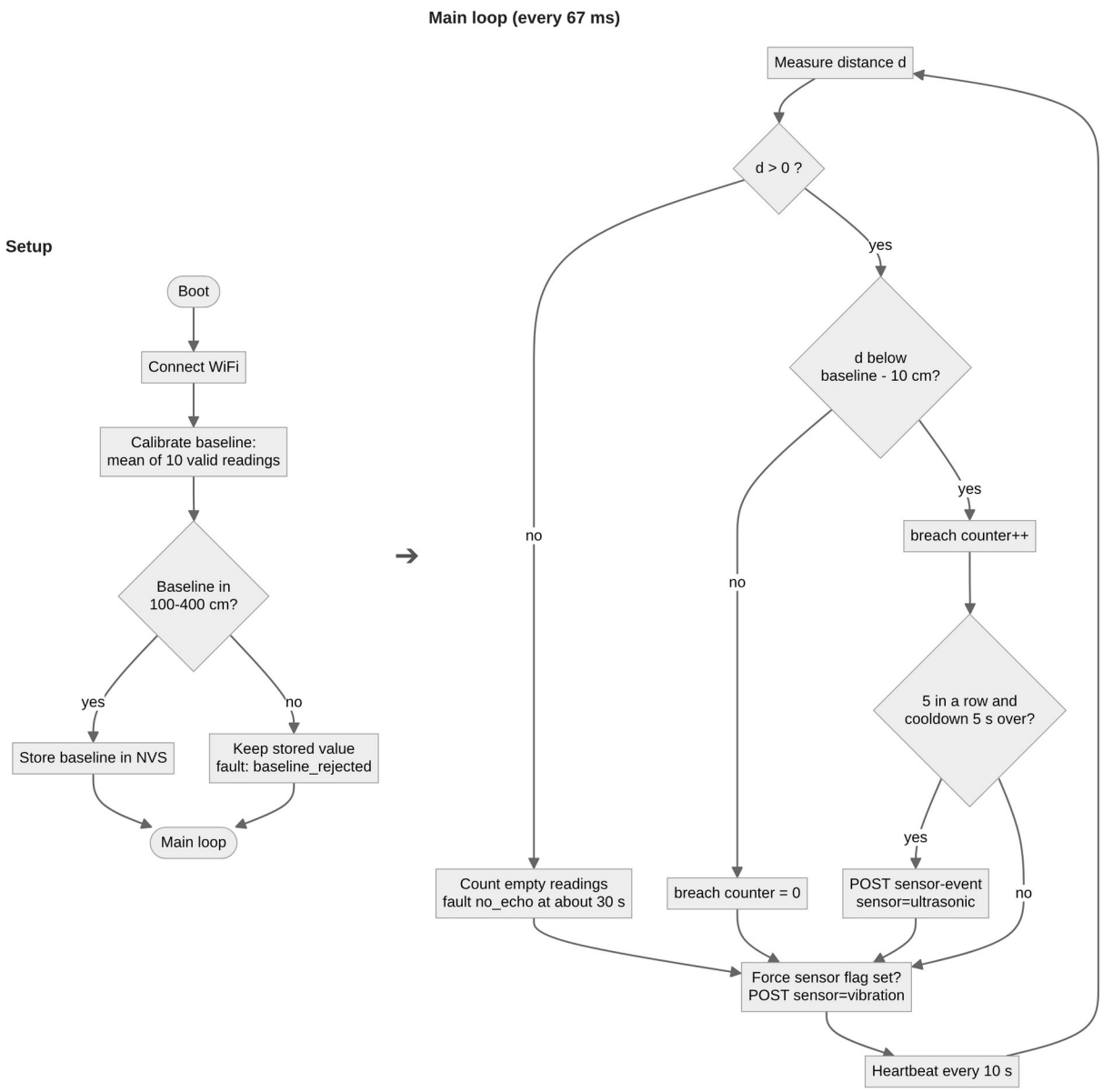
זיהוי חדירה — `loop()`. הלולאה דוגמת בקצב של כ-15 קריאות לשנייה (`SAMPLE_INTERVAL_MS = 67`). קריאה נחשבת חריגה כאשר:

$$d < d_{baseline} - \Delta, \Delta = 10\,cm$$

חדירה מדווחת רק לאחר **חמש חריגות רצופות** (REQUIRED_BREACHES = 5), כלומר לפחות כ-$5 \times 67\,ms \approx 0.34\,s$ של חריגה מתמשכת. קריאה תקינה אחת מאפסת את המונה, וכך רעש רגעי אינו מפעיל אזעקה. בנוסף מזוהה חיישן מת: רצף של כ-30 שניות של קריאות 0 (no_echo) מדווח כתקלה, מפני שחיישן מנותק מחזיר 0 לנצח וללא דיווח הוא נראה בריא.

```
404 void loop() {
405   float currentDistance = getDistance();
406 
407   // (0 מעל) אנחנו בודקים רק קריאות חוקיות
408   if (currentDistance > 0) {
409     // קריאה תקינה: אם היינו ברצף של אפסים, החיישן חי אחרי הכל.
410     consecutiveZeroReadings = 0;
411     if (activeFault == FAULT_NO_ECHO) clearFault();
412 
413     // אם המרחק קטן משמעותית מהמרחק לקיר ממול — כנראה שמישהו/משהו חוסם
414     if (currentDistance < (DEFAULT_DISTANCE - TOLERANCE)) {
415       consecutiveBreaches++; // מעלים את המונה
416       Serial.printf("Anomaly detected! Distance: %.1f cm (Count: %d)\n",
    currentDistance, consecutiveBreaches);
417 
418       // אם הגענו לרצף הנדרש - זו חדירה אמיתית ולא רעש אקראי
419       if (consecutiveBreaches >= REQUIRED_BREACHES) {
420         consecutiveBreaches = 0; // איפוס המונה
421 
422         // ...
425         if (millis() - lastAlertTime >= ALERT_COOLDOWN) {
426           Serial.println("ALARM! Confirmed intrusion.");
427           sendAlert("ultrasonic", "Ultrasonic%20sensor%20breached");
428           lastAlertTime = millis();
429         }
430       }
431     } else {
432       // הטווח חזר להיות תקין - מאפסים את מונה רצף החריגות
433       consecutiveBreaches = 0;
434     }
435   } else {
436     // ...
440     consecutiveZeroReadings++;
441     if (consecutiveZeroReadings == ZERO_READINGS_FAULT) {
442       Serial.printf("WARNING: %d consecutive empty readings (~%lus) — sensor looks
    dead.\n",
443                     consecutiveZeroReadings, ZERO_FAULT_MS / 1000);
444       setFault(FAULT_NO_ECHO);
445     }
446   }
```

קוד 3: לולאת הזיהוי הראשית של לוח האולטרה-סוני



איור 16: תרשים זרימה — לוח האולטרה-סוני

4.3.3 לוח הרעידות — פסיקה במקום דגימה

מודול SW-420 אינו "מחזיק" את הפין גבוה, אלא פולט צרור פולסים קצר (מיקרו-שניות עד מילי-שניות בודדות). בגרסה הראשונה נדגם הפין כל 50 ms (`digitalRead`), ולפיכך צרור באורך t_b נתפס רק אם הפין היה גבוה בדיוק ברגע הדגימה, בהסתברות של בערך:

$$P \approx \frac{t_b}{T_{sample}} = \frac{2\,ms}{50\,ms} = 4\,\%$$

כלומר מכה חזקה אחת עלולה לא לייצר שום אירוע, ובכך נכשלת המטרה היחידה של החיישן. לפיכך הוחלף המנגנון בפסיקה על הקצה העולה ([25] `attachInterrupt(..., RISING)`: שגרת השירות `onVibration()` מרימה דגל `volatile bool` בלבד, והלולאה הראשית צורכת אותו ומדווחת בכפוף ל-Cooldown. הפסיקה נרשמת עוד לפני חיבור ה-WiFi, כך שמכה בזמן ההתחברות נשמרת ומדווחת מיד כשהרשת זמינה. פונקציית הפסיקה מוגדרת `IRAM_ATTR`, מפני שקוד פסיקה חייב לשבת ב-RAM ולא ב-flash.

```
54  volatile bool vibrationLatched = false;
55
56  // ...
60  void IRAM_ATTR onVibration() {
61    vibrationLatched = true;
62  }
 ⋮
93    attachInterrupt(digitalPinToInterrupt(vibrationPin), onVibration, RISING);
 ⋮
166 void loop() {
167   // ...
171   if (vibrationLatched) {
172     vibrationLatched = false;
173     Serial.println("Vibration Detected! (Force sensor triggered)");
174
175     if (millis() - lastReportTime >= REPORT_COOLDOWN) {
176       sendAlert();
177       lastReportTime = millis();
178     }
179   }
180
181   if (millis() - lastHeartbeat >= HEARTBEAT_INTERVAL) {
182     sendHeartbeat();
183     lastHeartbeat = millis();
184   }
185
186   // ...
190   delay(10);
191 }
```

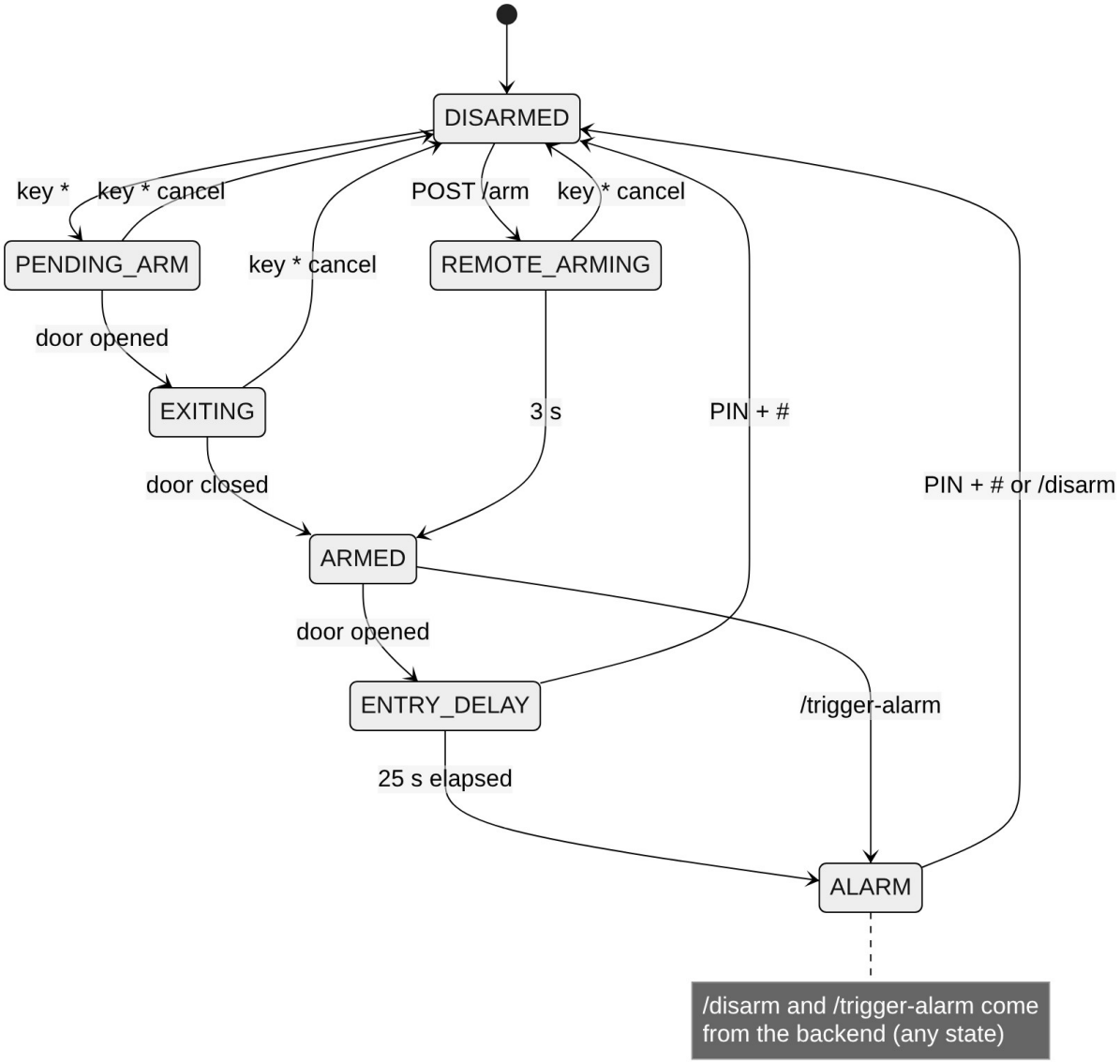
קוד 4: זיהוי רעידות באמצעות פסיקה

4.3.4 בקר הדלת והקודן

מכונת מצבים מקומית. הבקר מממש מכונת מצבים בת שבעה מצבים (איור 17, טבלה 6). המצבים PENDING_ARM ו-EXITING משקפים דריכה "מהקודן" (הדלת נפתחת, אדם יוצא, והדלת נסגרת), ואילו REMOTE_ARMING מייצג דריכה מהאפליקציה ללא צורך ביציאה פיזית.

טבלה 6: מצבי בקר הדלת ומעבריהם

מצב	מעבר	תנאי
DISARMED	→ PENDING_ARM	הקשת * כשהדלת סגורה
PENDING_ARM	→ EXITING	הדלת נפתחת
EXITING	→ ARMED	הדלת נסגרת (ומדווח `state-change`)
REMOTE_ARMING	→ ARMED	3 שניות לאחר פקודת `/arm` (ומדווח)
ARMED	→ ENTRY_DELAY	הדלת נפתחת (ומדווח `entry-delay`)
ENTRY_DELAY	→ DISARMED	PIN נכון + #
ENTRY_DELAY	→ ALARM	חלפו 25 שניות (ומדווח חדירה)
ALARM	→ DISARMED	PIN נכון + #
כל מצב	→ DISARMED	פקודת `/disarm` מהבקנד
כל מצב	→ ALARM	פקודת `/trigger-alarm` מהבקנד



איור 17: מכונת המצבים של בקר הדלת

```
783     case ENTRY_DELAY:
784       {
785         unsigned long elapsed = millis() - delayStartTime;
786 
787         // 1. Check if they ran out of time
788         if (elapsed >= ENTRY_TIME_LIMIT) {
789           currentState = ALARM;
790           enteredPin = "";
791           lastBeepTime = millis();
792           Serial.println("\n*** ALARM TRIGGERED! SIRENS SOUNDING! ***");
793           reportDoorAlert();
794         }
795         else {
796           // 2. Handle keypad to disarm
797           if (input == '#') {
798             Serial.println();
799             if (enteredPin == secretPin) {
800               currentState = DISARMED;
801               enteredPin = "";
802               silenceBuzzers();
803               Serial.println("Correct PIN. System DISARMED.");
804               reportStateChange(false);
805               // ...
811               break;
812             } else {
813               enteredPin = "";
814               Serial.println("Wrong PIN. Keep trying!");
815             }
816           }
817 
818           // 3. Accelerating Beep Logic WITH SILENT DELAY
819           if (elapsed < SILENCE_BEFORE_BEEP) {
820             // Do nothing yet, keep it silent
821             silenceBuzzers();
822           }
823           else {
824             // The silent time has passed, start the warning beeps!
825             // Map the remaining time (from when silence ends to the time limit)
826             int currentOffDuration = map(elapsed, SILENCE_BEFORE_BEEP,
    ENTRY_TIME_LIMIT, 1500, 60);
827             unsigned long timeSinceLastChange = millis() - lastBeepTime;
828 
829             if (isBuzzerCurrentlyOn) {
830               if (timeSinceLastChange >= BEEP_ON_DURATION) {
831                 silenceBuzzers();
832                 lastBeepTime = millis();
833               }
834             } else {
835               if (timeSinceLastChange >= currentOffDuration) {
836                 digitalWrite(BUZZER_1_PIN, HIGH);
837                 digitalWrite(BUZZER_2_PIN, HIGH);
838                 isBuzzerCurrentlyOn = true;
839                 lastBeepTime = millis();
840               }
841             }
842           }
843         }
844       }
845       break;
```

קוד 5: מכונת המצבים של בקר הדלת

אזהרה מתגברת בחלון הנטרול. לאחר פתיחת דלת על מערכת דרוכה עומדות לרשות המשתמש 25 שניות להקיש קוד. בשתי השניות הראשונות הבקר שותק, ולאחר מכן מתחיל לצפצף (150

ms בכל צפצוף), כאשר זמן ההשהיה בין צפצופים מתקצר ליניארית מ-1500 ms ל-60 ms לפי הפונקציה `map(elapsed, 2000, 25000, 1500, 60)`. כך המשתמש מבין מהר ככל שהזמן אוזל. במצב ALARM הזמזמים מופעלים לסירוגין בחילוף של 100 ms (אחד דולק ואחד כבוי) לצליל חד ומטריד.

סריקת הקודן — `getPhysicalKey()`. הפונקציה מעבירה שורה אחת בכל פעם ל-LOW (שאר השורות HIGH), ובודקת איזו עמודה נקראת LOW; הצטלבות השורה והעמודה היא המקש שנלחץ. מניעת קריאות כפולות מתבצעת בשני תנאים: המקש חייב להשתחרר (`keyWasDown`) לפני רישום נוסף, ונדרש מרווח מינימלי של 200 ms בין לחיצות. בלי זה לחיצה בודדת הייתה נקראת עשרות פעמים בלולאה חסרת השהיה.

```
const unsigned long KEYPAD_DEBOUNCE_MS = 200;
unsigned long lastKeyTime = 0;
bool keyWasDown = false;

char getPhysicalKey() {
  char found = '\0';
  for (int r = 0; r < 4 && found == '\0'; r++) {
    for (int rr = 0; rr < 4; rr++) {
      digitalWrite(ROW_PINS[rr], rr == r ? LOW : HIGH);
    }
    delayMicroseconds(50); // let the line settle before sampling columns
    for (int c = 0; c < 3; c++) {
      if (digitalRead(COL_PINS[c]) == LOW) {
        found = KEYPAD_LAYOUT[r][c];
        break;
      }
    }
  }
  for (int rr = 0; rr < 4; rr++) digitalWrite(ROW_PINS[rr], HIGH); // back to idle

  if (found == '\0') {
    keyWasDown = false;
    return '\0';
  }
  if (keyWasDown || millis() - lastKeyTime < KEYPAD_DEBOUNCE_MS) {
    return '\0'; // still holding the previous press, or debounce window open
  }
  keyWasDown = true;
  lastKeyTime = millis();
  return found;
}
```

קוד 6: סריקת קודן מטריצה עם ניפוי הקפצות (Debounce)

דיווח לבקנד ב-Task נפרד. בקר הדלת הוא היחיד שגם מדווח וגם מאזין (Web Server על פורט 80). כל דיווח סינכרוני חסם את `loop()` ואיתו את `server.handleClient()`, כך שבזמן הדיווח הלוח לא ענה לפקודה `trigger-alarm/` שהבקנד שולח בעת חדירה שדווחה מלוח אחר. התוצאה

האפשרית: אזעקה שקטה במהלך חדירה אמיתית. הפתרון: `loop()` מניח הודעה קטנה בתור FreeRTOS [21] (עומק 8) ומחזיר מיד, ו-Task ייעודי (`reporterTask`, נעוץ לליבה 0, מחסנית 8 KB) מבצע את השליחה בפועל. תור מלא מפיל הודעות במקום לצבור התפרצות שתגיע בבת אחת כשהבקנד יחזור, ו-Heartbeat ממילא נשלח מחדש כל 10 שניות. אם יצירת התור נכשלה, הבקר חוזר לשליחה מבוססת `loop()` במקום לדווח כלום.

```
500 void reporterTask(void* param) {
501   ReportMessage message;
502   for (;;) {
503     // Blocks indefinitely; this task consumes no CPU while idle.
504     if (xQueueReceive(reportQueue, &message, portMAX_DELAY) != pdTRUE) continue;
505     switch (message.kind) {
506       case REPORT_STATE_CHANGE: sendStateChangeNow(message.armed); break;
507       case REPORT_ENTRY_DELAY:  sendEntryDelayNow();               break;
508       case REPORT_DOOR_ALERT:   sendDoorAlertNow();                break;
509       case REPORT_HEARTBEAT:    sendHeartbeatNow();                break;
510     }
511   }
512 }
513
514 // ...
521 void enqueueReport(ReportKind kind, bool armed = false) {
522   ReportMessage message = {kind, armed};
523   // ...
526   if (reportQueue == nullptr) {
527     switch (kind) {
528       case REPORT_STATE_CHANGE: sendStateChangeNow(armed); break;
529       case REPORT_ENTRY_DELAY:  sendEntryDelayNow();       break;
530       case REPORT_DOOR_ALERT:   sendDoorAlertNow();        break;
531       case REPORT_HEARTBEAT:    sendHeartbeatNow();        break;
532     }
533     return;
534   }
535   if (xQueueSend(reportQueue, &message, 0) != pdTRUE) {
536     Serial.println("Report queue full — dropping a report (is the backend up?)");
537   }
538 }
539
540 void reportStateChange(bool armed) { enqueueReport(REPORT_STATE_CHANGE, armed); }
541 void reportEntryDelay()            { enqueueReport(REPORT_ENTRY_DELAY); }
542 void reportDoorAlert()             { enqueueReport(REPORT_DOOR_ALERT); }
543 void sendHeartbeat()               { enqueueReport(REPORT_HEARTBEAT); }
```

קוד 7: דיווח אסינכרוני באמצעות תור FreeRTOS

פקודות מהבקנד ואבטחתן. מטפלי `/trigger-alarm`, `/disarm`, `/arm` ו-`/keypad-pin` מתחילים בבדיקת `authorized()`. הבדיקה הכרחית משום ש-`/disarm` משתיק אזעקה פעילה, וכל מי שגילה את כתובת ה-IP של הלוח היה יכול לנטרל את המערכת בבקשה אחת בלי אימות. `/`

`disarm` נענה לפני שהדיווח לבקנד נשלח, כדי שהגבלת הזמן של 3 שניות בבקנד לא תיחשב כישלון (`"hardware: false"`) אם דיווח הנטרול נתקע ברשת.

קוד הקודן. קוד הנטרול הוא 4–8 ספרות (המקשים `*` ו-`#` שמורים לדריכה ולאישור, ולכן אינם יכולים להיות חלק מקוד). הקוד נשמר בזיכרון [19] NVS, שורד כיבוי ואתחול, ואינו מודפס לטרמינל כדי שלא יזלוג למי שיש לו גישה פיזית לחיבור ה-USB. `KEYPAD_PIN` ב-`secrets.h` משמש רק ברירת מחדל ללוח ש-NVS שלו מעולם לא נכתב.

ESP32-CAM 4.3.5

ללוח המצלמה שני תפקידים עצמאיים, המבוצעים בשני נתיבים נפרדים:

דחיפת פריימים לזיהוי פנים — `sendFrame()`. כל 500 ms (`CAPTURE_INTERVAL_MS`, כלומר 2 פריימים בשנייה) נלכד פריים JPEG ונשלח כ-`multipart/form-data` (שדה `photo`) ל-`/camera/recognize`. גוף הבקשה נבנה ידנית, מפני שב-`HTTPClient` של ESP32 אין תמיכה מובנית ב-multipart. החיבור נשאר פתוח בין פריימים (`setReuse(true)`), כדי שלא ישולם חיבור TCP חדש בכל פריים. בנוסף נבנו מראש מחרוזות הבקשה (URL, כותרות), כדי למנוע הקצאות זיכרון חוזרות בגודל קטן על ערימה מוגבלת, שנוטות להתפתח לכשל מתמשך אחרי שעות ריצה.

ניהול כשלים — `noteSendResult()`. פריים שנכשל (למשל 401 או 500) מפעיל המתנה מעריכית: מ-500 ms, בהכפלה בכל כישלון, ועד 30 שניות. ללא כך, אותו קובץ JPEG בן עשרות KB היה נשלח שוב ושוב פעמיים בשנייה לבקנד שאינו יכול לקבלו, על אותו רדיו שדרכו עוברים גם דיווחי האזעקה. לאחר 40 כשלים רצופים הלוח מתאתחל (`esp_restart`), הערכה: כ-20 דקות ללא שרת, אך רק כ-20 שניות כשהכשל מקומי (חיישן תקוע או חוסר זיכרון). ההבחנה מכוונת: איתחול אינו יכול לתקן שרת של מישהו אחר, אך כן מתקן את הלוח עצמו.

```
  cameraHttp.begin(CAMERA_UPLOAD_URL);
  cameraHttp.setReuse(true);
  cameraHttp.addHeader("X-Device-Key", DEVICE_KEY);
  cameraHttp.addHeader("Content-Type", MULTIPART_CONTENT_TYPE);

  int code = cameraHttp.POST(body, totalLen);
  free(body);
  cameraHttp.end();  // keeps the TCP connection open — see setReuse() above

  if (code == 401) {
    Serial.println("Frame REJECTED (401) — DEVICE_KEY in secrets.h doesn't match the backend's.");
  } else if (code < 0) {
    Serial.printf("Error sending frame: %s\n", cameraHttp.errorToString(code).c_str());
  } else if (code < 200 || code >= 300) {
    Serial.printf("Backend refused the frame: HTTP %d\n", code);
  }
  // ...

  return code;
}
⋮
void noteSendResult(int result) {
  if (result >= 200 && result < 300) {
    consecutiveFailures = 0;
    sendBackoffMs = 0;
    return;
  }

  consecutiveFailures++;

  if (result != SEND_LOCAL_FAILURE) {
    unsigned long next = (sendBackoffMs == 0) ? BACKOFF_MIN_MS : sendBackoffMs * 2;
    sendBackoffMs = (next > BACKOFF_MAX_MS) ? BACKOFF_MAX_MS : next;
    Serial.printf("Backing off %lums (%d frames in a row failed)\n", sendBackoffMs, consecutiveFailures);
  }

  if (consecutiveFailures >= FAILURE_RESTART_THRESHOLD) {
    Serial.printf("FATAL: %d frames in a row failed — restarting.\n", consecutiveFailures);
    Serial.flush();  // esp_restart() doesn't drain the UART on its way out
    delay(1000);
    esp_restart();
  }
}
```

קוד 8: שליחת פריים וניהול כשלים בלוח המצלמה

הגשת זרם וידאו חי — `streamHandler()`. שרת ה-MJPEG ממומש באמצעות `esp_http_server` ולא ב-`WebServer` של Arduino [7]. `httpd_start()` מריץ את המטפל ב-Task משלו, כך שלולאת הזרם האינסופית אינה חוסמת את `loop()`. עם `WebServer`, כל עוד `loop()` היה חסום ושליחת הפריימים לזיהוי פנים הייתה נעצרת לחלוטין. אימות הגישה מתבצע באמצעות `X-Device-Key` (הכותרת שהבקנד שולח) או `?key=` (לדפדפן או

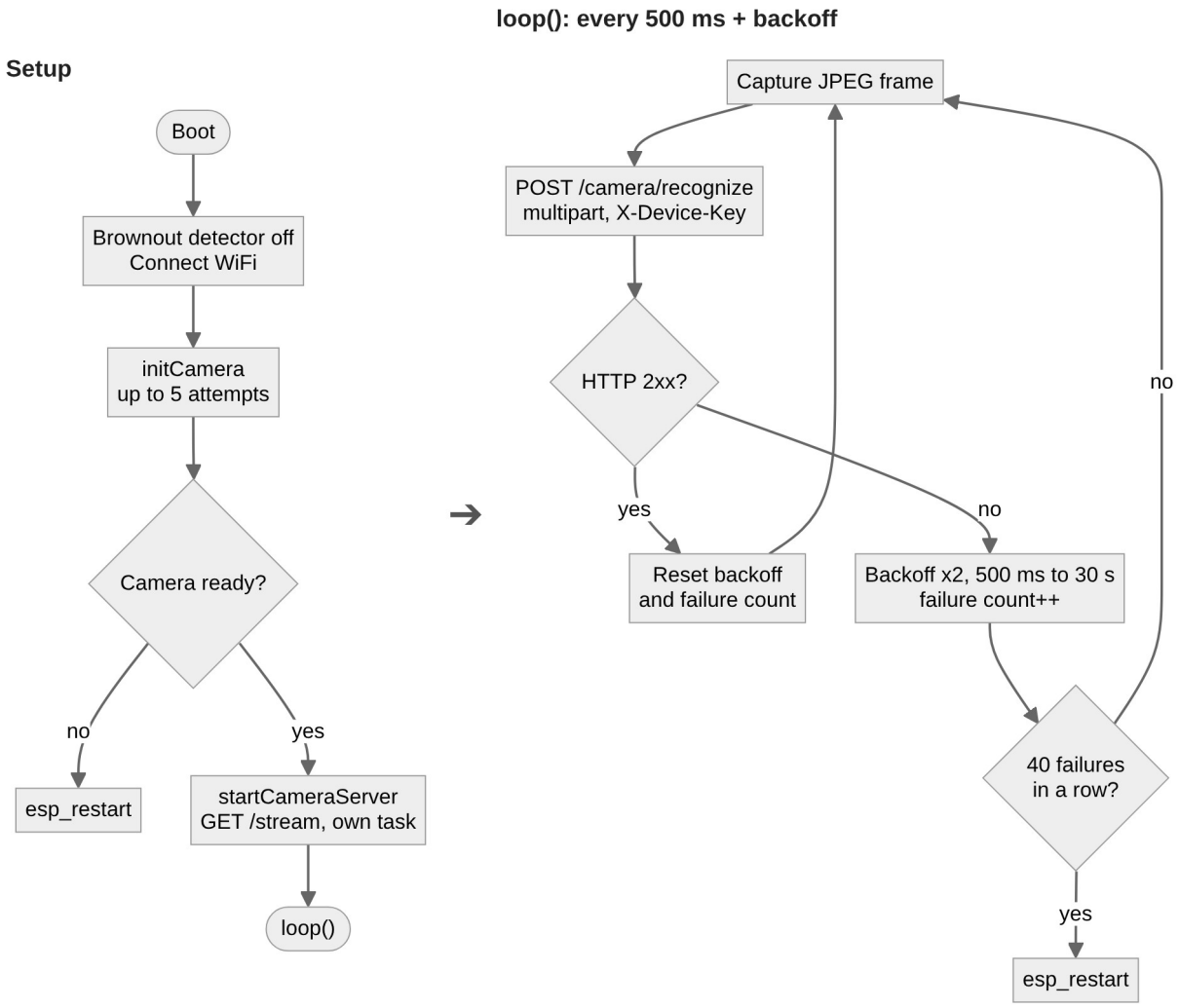
נגן שאינם יכולים לשלוח כותרת). `initCamera()` מגדיר `CAMERA_GRAB_LATEST` עם שני חוצצים (`fb_count = 2`) בנוכחות PSRAM, כך ששני צרכנים בו-זמניים מקבלים תמיד את הפריים העדכני ואינם צוברים השהיה; בלי PSRAM מופעלת חלופה ברזולוציה נמוכה (CIF) עם חוצץ יחיד [20].

```
  if (psramFound()) {
    config.frame_size = FRAMESIZE_VGA;
    config.jpeg_quality = 12;
    config.fb_count = 2;
    config.fb_location = CAMERA_FB_IN_PSRAM;
    config.grab_mode = CAMERA_GRAB_LATEST;
  } else {
    config.frame_size = FRAMESIZE_CIF;
    config.jpeg_quality = 15;
    config.fb_count = 1;
    config.fb_location = CAMERA_FB_IN_DRAM;
    config.grab_mode = CAMERA_GRAB_LATEST;
  }
  ⋮
esp_err_t streamHandler(httpd_req_t* req) {
  if (!authorized(req)) {
    httpd_resp_send_err(req, HTTPD_401_UNAUTHORIZED, NULL);
    return ESP_FAIL;
  }

  esp_err_t res = httpd_resp_set_type(req, STREAM_CONTENT_TYPE);
  if (res != ESP_OK) return res;

  char partHeader[64];
  while (true) {
    camera_fb_t* fb = esp_camera_fb_get();
    if (!fb) {
      res = ESP_FAIL;
    } else {
      res = httpd_resp_send_chunk(req, STREAM_BOUNDARY, strlen(STREAM_BOUNDARY));
      if (res == ESP_OK) {
        size_t hlen = snprintf(partHeader, sizeof(partHeader), STREAM_PART_HEADER, fb-
>len);
        res = httpd_resp_send_chunk(req, partHeader, hlen);
      }
      if (res == ESP_OK) {
        res = httpd_resp_send_chunk(req, (const char*)fb->buf, fb->len);
      }
      esp_camera_fb_return(fb);
    }
    // ...
    if (res != ESP_OK) break;
  }
  return res;
}
```

קוד 9: שרת הזרם וקביעות המצלמה



איור 18: תרשים זרימה — ESP32-CAM

4.4 מימוש הבקנד (Python / FastAPI)

4.4.1 מבנה הבקנד

הבקנד מחולק למודולים לפי תפקיד:

טבלה 7: מודולי הבקנד

מודול	תפקיד
`main.py`	הגדרת ה-routes, האימות לכל route ו-WebSocket
`state.py`	מחלקת `AppState`: מצב האזעקה, יומן, הגדרות, טיימרים ושמירה לדיסק
`models.py`	מבני נתונים (Pydantic) התואמים 1:1 את הלקוח ב-Dart

מודול	תפקיד
`security.py`	גיבוב סיסמה (scrypt), אסימוני סשן, פתרון מפתח מכשיר
`esp32_client.py`	פקודות יוצאות לבקר הדלת
`face_recognition_service.py`	זיהוי פנים וזיהוי חיות
`notifier.py`	התראות ntfy
`ip_check.py`	בדיקת התאמה בין IP המחשב לכתובות שצרובות בלוחות

כל מצב האזעקה נשמר בזיכרון בתהליך יחיד (event loop אחד), כאשר הגדרות, פנים, יומן ופרטי אימות נשמרים בקובץ `state.json` בכתיבה אטומית. `alarm_state` אינו נשמר בכוונה: בקנד שהופעל מחדש אינו יודע אם הבית דרוך, ולכן עולה כמנוטרל ומחכה לדיווח הבא של בקר הדלת, במקום להציג מצב ישן שאיש לא אימת.

4.4.2 מכונת המצבים והלוגיקה המרכזית

מצב האזעקה נקבע על ידי `AlarmState`: חמישה ערכים — `disarmed`, `transitional`, `armed`, `entryDelay`, `triggered`. הפונקציות המרכזיות ב-`AppState`:

`arm()` ו-`arm_finish_arming()`. פועלת רק מ-`disarmed`, כך שלחיצה כפולה באפליקציה אינה מאפסת את ספירת היציאה. היא עוברת מיד ל-`transitional` ומשדרת, ומתזמנת משימה אסינכרונית שממתינה 3 שניות. לאחר ההמתנה נבדק שוב שהמצב עדיין `transitional` לפני המעבר ל-`armed`: ללא בדיקה זו, `disarm()` שמתרחש בין שידור המצב לבין שמירת המשימה (חלון קצר שבו אין עדיין משימה לבטל) היה מחזיר את הבית למצב דרוך שלוש שניות אחר כך, בעוד שהחומרה מנוטרלת.

`apply_bypass()` — מצב Home. במצב Home חיישן הרעידות עקוף (`bypassed=True`), כך שאפשר לשהות בבית כשהמערכת דרוכה. הלוחות אינם יודעים זאת ומדווחים כרגיל; `trigger_alert` הוא שמתעלם מדיווח של חיישן עקוף. מכיוון שחיישן הכוח בלוח 1 מדווח כ-`vibration`, גם הוא עקוף במצב Home.

`report_state_from_esp32()`. בקר הדלת הוא הסמכות לדריכה, ולכן דיווחו גובר על המצב המקומי, גם כשהוא מגיע באמצע ספירת היציאה (ואז הטיימר המקומי מבוטל). חריג אחד: בקר שמדווח "דרוך" בעת אזעקה פעילה — הדיווח מתעלם ורק נרשם ביומן, מפני שבלעדיו מעבר `triggered → armed` מאפס את סימון החיישן שגרם לאזעקה ומעלים את הבאנר האדום

מהאפליקציה ללא `disarmed` שיסביר זאת. רק נטרול אנושי (קודן, אפליקציה או פנים) יוצא ממצב `triggered`.

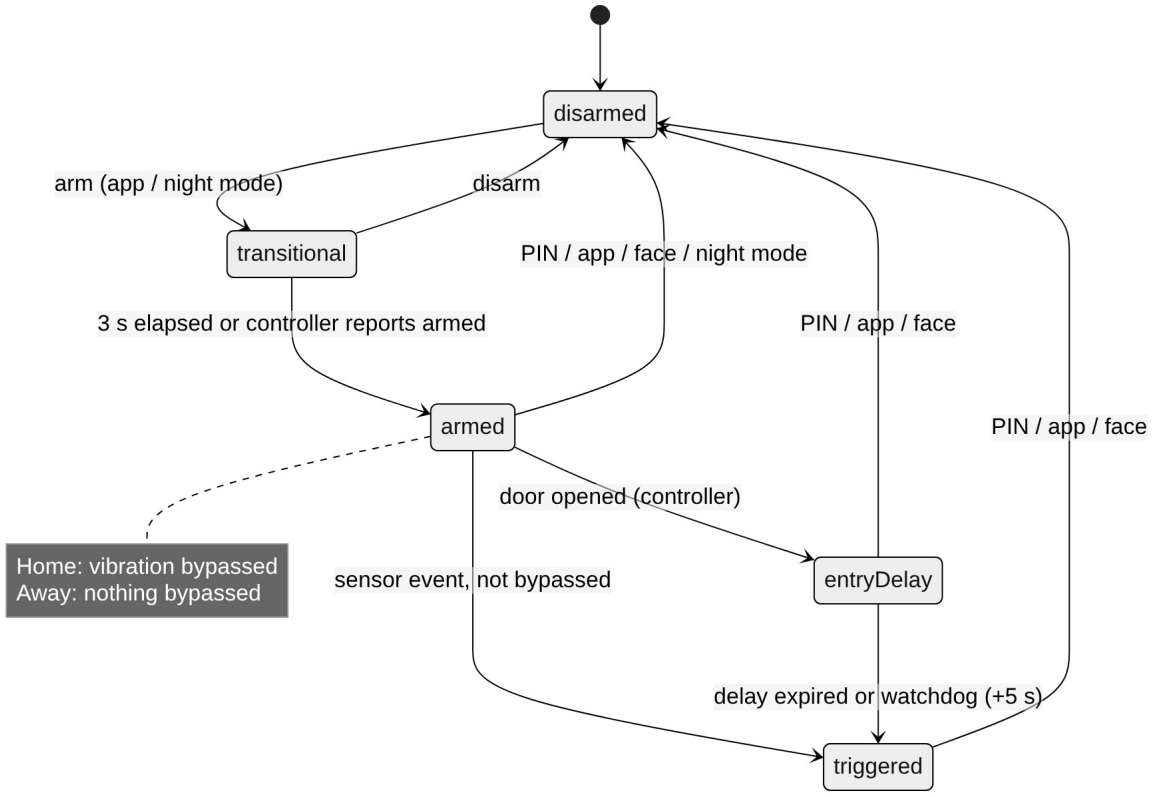
`begin_entry_delay()` ו-`entry_delay_watchdog()`. כשבקר הדלת מדווח על תחילת ספירת הנטרול, הבקנד עובר ל-`entryDelay` ושומר את מועד הסיום כדי להציג באפליקציה ספירה לאחור. הספירה האמיתית מתבצעת בבקר, שנמצא בתוך הבית שאליו נכנס הפורץ. לפיכך הבקנד מפעיל משימת "כלב שמירה" (Watchdog): אם עברו משך הנטרול ועוד `ENTRY_DELAY_GRACE = 5` שניות והמצב עדיין `entryDelay`, הבקנד מעלה אזעקה בעצמו. ללא כך, ניתוק מתח או WiFi של הלוח בין פתיחת הדלת לפקיעת הספירה היה משאיר את המערכת לנצח ב-`entryDelay`, ללא אזעקה, ללא התראה, ועם ספירה שלא מגיעה לאפס. כל דרך לגיטימית לצאת מהמצב מבטלת את המשימה (`_clear_alarm_timers`).

`trigger_alert()`. הפונקציה מרכזת את ההחלטה:

1. מתעלמת אם המערכת אינה `armed` או `entryDelay` (כולל `transitional`, שבו חיישנים עשויים להפעיל בצדק תוך כדי יציאה).
2. מתעלמת אם החיישן המדווח עקוף.
3. מסמנת את החיישן כ-`ok=False`, עוברת ל-`triggered`, רושמת ליומן ושומרת לדיסק.
4. שולחת התראת ntfy **ללא המתנה** (`notifier.notify` אינה קורוטינה — ראה 4.4.6) ומתזמנת הסלמה.
5. משדרת את המצב לכל האפליקציות **לפני** פנייה לחומרה.
6. שולחת `/trigger-alarm` לבקר הדלת (למעט כשהדלת עצמה היא המקור, כי הזמזמים כבר פועלים), עם ניסיון חוזר אחד לאחר 0.5 שניות, מפני שזהו הכישלון היחיד שבו זמזם שלא מופעל נראה מתוך הבית כמו היעדר חדירה.

```
1163     async def trigger_alert(self, source: SensorType | None, reason: str) -> None:
1164         """  ...  """
1172         if self.alarm_state not in (AlarmState.armed, AlarmState.entryDelay):
1173             # ...
1181             return
1182 
1183         # ...
1187         if source is not None:
1188             reporting_sensor = next((s for s in self.sensors if s.type == source),
     None)
1189             if reporting_sensor is not None and reporting_sensor.bypassed:
1190                 return
1191 
1192             self.sensors = [
1193                 SensorReading(type=s.type, label=s.label, ok=False,
     bypassed=s.bypassed)
1194                 if s.type == source
1195                 else s
1196                 for s in self.sensors
1197             ]
1198         self.alarm_state = AlarmState.triggered
1199         self._log(f"ALERT — {reason}", EventCategory.alert)
1200         self._persist()  # an alert is exactly the event worth surviving a restart
1201         # ...
1204         notifier.notify(
1205             self.ntfy_server,
1206             self.ntfy_topic,
1207             title="HomeGuard alarm",
1208             message=reason,
1209         )
1210         self._start_escalation(reason)
1211 
1212         # ...
1220         await self._broadcast()
1221 
1222         # ...
1228         if source != SensorType.door:
1229             from . import esp32_client  # deferred: esp32_client imports `state`
1230 
1231             await esp32_client.send_trigger_alarm()
```

קוד 10: טיפול בהתראה בבקנד



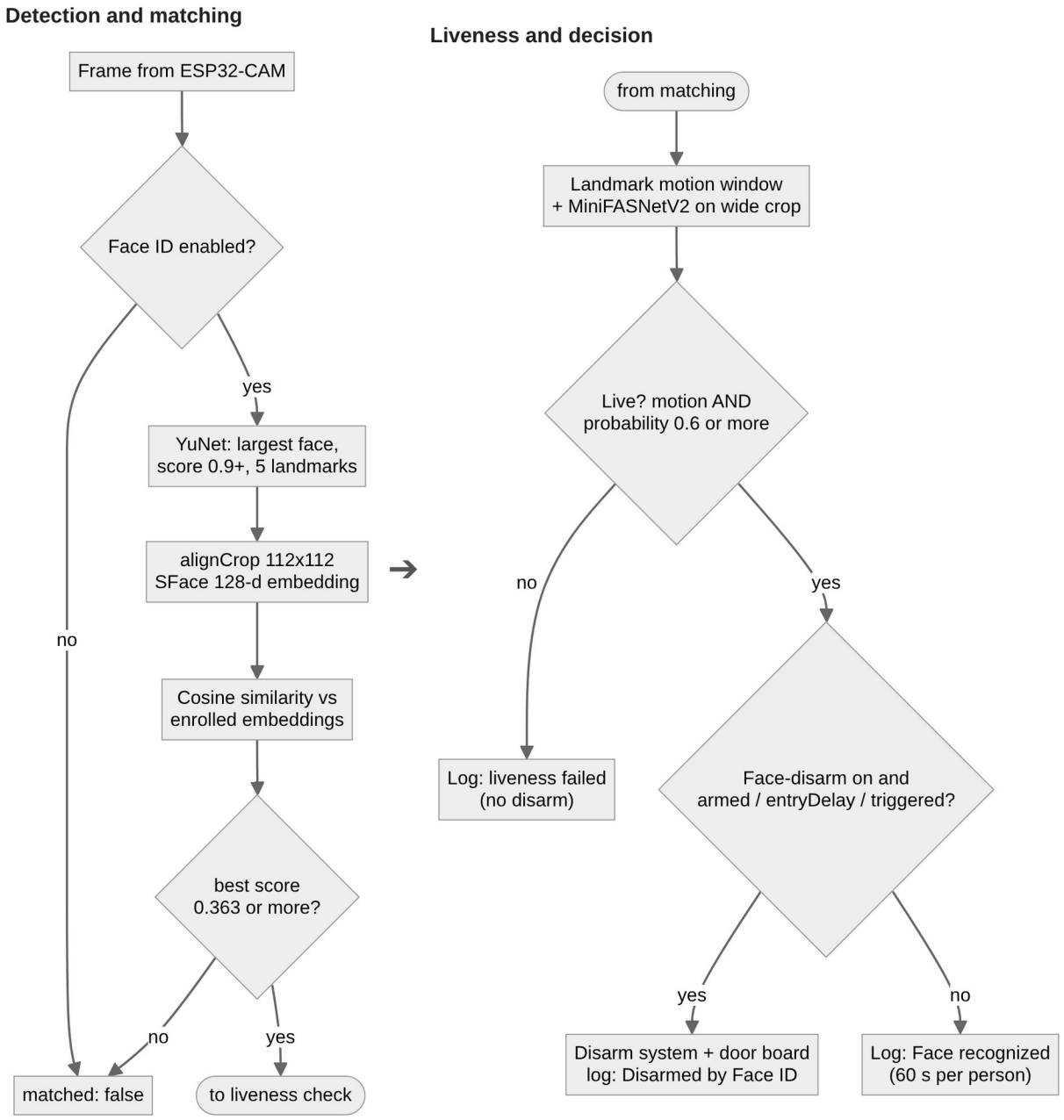
איור 19: מכונת המצבים של הבקנד

הסלמה ו-Night Mode. אם האזעקה לא נוקתה תוך `police_call_delay_seconds`, נשלחת התראה דחופה שנייה (`priority=urgent`) המציינת למי להתקשר (`escalate_after_delay_`). חיוג טלפוני אוטומטי אינו ממומש, כי הוא מצריך ספק טלפוניה, ולכן ההסלמה היא התראה בלבד. Night Mode (`_night_mode_tick`) נבדק פעם בדקה ופועל רק על השעה העגולה, כדי שנטרול מכוון ב-02:30 לא יבוטל דקה אחר כך. הוא דורך אוטומטית רק מ-`disarmed` ומנטרל רק מ-`armed`, ולעולם אינו מנטרל אזעקה חיה.

4.4.3 תקשורת עם בקר הדלת

הפקודות לבקר מבוצעות ב-`esp32_client.py` באמצעות `httpx.AsyncClient` עם מגבלת זמן של 3 שניות. כל פקודה מחזירה ערך בוליאני (הגיע / לא הגיע) ולא חריגה, וה-routes `/arm` ו-`/disarm` מחזירים `{"ok": true, "hardware": <bool>}`. זה מונע את ההטעיה שבה לחיצה על DISARM ללא נתיב לבקר נראית כנטרול שהצליח. פעולות `arm/` ו-`disarm/` אינן אטומיות מול החומרה: המצב המקומי מתעדכן תחילה, והדיווחים של בקר הדלת (`state-change`) הם שמיישרים קו בסופו של דבר.

4.4.4 זיהוי פנים וזיהוי חיות



איור 20: צינור זיהוי הפנים בבקנד

רישום וזיהוי. הבקנד טוען שלושה קבצי ONNX דרך `opencv-contrib-python`: `yunet.onnx` (גילוי), `sface.onnx` (זיהוי זהות) ו-`minifasnet_v2.onnx` (זיהוי חיות). בחירה זו נעשתה במכוון על פני `insightface` ו-`face_recognition`, שדורשים כלי קומפילציה נייטיביים (MSVC/CMake) ב-Windows, כאב שהפרויקט כבר שילם עליו ימים. אם קבצי ה-ONNX חסרים, השירות נופל חזרה אוטומטית ל-Haar Cascade + LBPH ומתריע בלוג.

תהליך הזיהוי על פריים (`DnnBackend.recognize_`):

1. YuNet מזהה את הפנים ברף ביטחון `DETECT_SCORE_THRESHOLD = 0.9`, והגדול ביותר נבחר כנושא. הרף מחמיר במכוון, מפני שזיהוי שווא בעת רישום נכתב כזהות של אדם.
2. `alignCrop` מישר את הפנים לפי נקודות הציון לחתך קנוני בגודל 112×112, כך שהטיית ראש משנה את התיאור רק מעט.
3. SFace מפיק וקטור תיאור באורך 128 (a), והוא מושווה לכל וקטורי התמונות המאושרות (b) באמצעות דמיון קוסינוס (נוסחה בסעיף 3.5.2). נבחר הציון הטוב ביותר בין כל התמונות של כל אדם (ולא ממוצע), כך שתמונה חזיתית טובה אחת מספיקה.
4. התאמה מתקבלת אם $\cos(\theta) \geq 0.363$ — הסף ש-OpenCV מפרסם עבור מודל זה, שכויל על מאגר LFW [12], ואינו מספר שנקבע בפרויקט.

זיהוי חיות — שתי בדיקות בלתי תלויות. שתיהן חייבות לעבור כדי שזיהוי פנים יורשה לנטרל (`is_live`):

- **בדיקת תנועה (`is_live_landmark_sequence`).** נאספות נקודות הציון של 5 הנקודות בחלון של 2.5 שניות (לפחות 3 דגימות), מנורמלות ביחס למסגרת הפנים כדי להיות בלתי תלויות במרחק. תזוזה ממוצעת מחושבת כך:

$$J = \frac{1}{2N} \sum_{i=1}^{N} \left[\left(\max x_i - \min x_i \right) + \left(\max y_i - \min y_i \right) \right] \geq 0.008$$

כאשר $N = 5$ מספר הנקודות. פנים חיות מפגינות תזוזה זעירה (נשימה, ניעה, מצמוץ); תמונה אחוזה בקשיחות — כמעט אפס. הבדיקה נכשלת כלפי הבטוח: פחות משלוש דגימות נחשבות "לא הוכח כחי". - **מודל (`is_live_spoof_logits`) MiniFASNetV2.** חתך רחב פי 4 מגודל הפנים (כולל מסגרת נייר, שפת מסך או ברק) מוזן למודל ברזולוציה 80×80, והפלט (שלושה Logits) עובר Softmax. התמונה מוכרת כאמיתית רק אם ההסתברות של מחלקה 1 ("אמיתי") מגיעה ל-0.6 לפחות, ולפיכך מבטיחה גם שזו המחלקה המנצחת. בהיעדר קובץ המודל הבדיקה נכשלת כלפי הבטוח.

הבדיקות משלימות זו את זו: תמונה מנופפת עוברת את בדיקת התנועה (רעד יד), אך נושאת את חתימת המרקם של הדפסה או מסך שאליה המודל אומן; ומנגד, הקרנת וידאו עשויה להתעתע בבדיקת התנועה. עם זאת, שני הספים (0.008 ו-0.6) הם הערכות ראשוניות שנגזרו מניתוח ולא נמדדו מול מערך ניסיונות הונאה מסודר, ולכן יש להתייחס אליהם כמעורבים בכיול.

```
534     def recognize(self, image: np.ndarray) -> str | None:
535         """  ...  """
549         if not self._embeddings:
550             return None
551         detected = self._detect_and_align(image)
552         if detected is None:
553             return None
554         face_row, aligned = detected
555 
556         probe = self._recognizer.feature(aligned)
557         best_id: str | None = None
558         best_score = -1.0
559         for face_id, features in self._embeddings.items():
560             for feature in features:
561                 score = self._recognizer.match(
562                     probe, feature, cv2.FaceRecognizerSF_FR_COSINE
563                 )
564                 if score > best_score:
565                     best_score = score
566                     best_id = face_id
567 
568         if best_score < MATCH_SIMILARITY_THRESHOLD:
569             return None
570         self._record_landmark_sample(best_id, face_row)
571         self._spoof_verdicts[best_id] = self._is_real_face(image, face_row)
572         return best_id
  ⋮
645     def is_live(self, face_id: str) -> bool:
646         """  ...  """
660         self._prune_landmark_history(face_id)
661         history = self._landmark_history.get(face_id)
662         if not history:
663             return False
664         if not is_live_landmark_sequence([landmarks for _, landmarks in history]):
665             return False
666         return self._spoof_verdicts.get(face_id, False)
```

קוד 11: זיהוי פנים ובדיקות חיות

הלוגיקה סביב ההתאמה — `log_face_recognized()`. נטרול לפי פנים כבוי כברירת מחדל ודורש הפעלה מפורשת במסך המצלמה. נטרול מותר רק ממצבי `armed`, `triggered` או `entryDelay`, ורק אם בדיקת החיות עברה. התאמה שהייתה מנטרלת אך נכשלה בבדיקת החיות נרשמת ביומן בשורה נפרדת ("Face recognized but liveness check failed"), מפני שזו הצורה שבה ניסיון הונאה נראה. דיווח על אותו אדם מוגבל לפעם ב-60 שניות (`FACE_LOG_COOLDOWN`), שכן המצלמה שולחת שני פריימים בשנייה, ובלי ההגבלה היומן בן 200 הרשומות היה מתמלא בפחות משתי דקות. נטרול תמיד נרשם, ללא קשר להגבלה.

עיבוד מחוץ ל-Event Loop. הזיהוי (כ-21 ms של OpenCV לפריים לפי מדידה שנרשמה בקוד) מורץ ב-`asyncio.to_thread` תחת נעילה (`recognizer_lock_`), כדי שרישום פנים חדשות

במקביל לא יבנה מחדש את המודל מתחת לזיהוי פעיל, וכדי שה-Event Loop היחיד, המשרת גם את ה-WebSocket ואת דיווחי הבקרים, לא ייחסם.

4.4.5 העברת זרם הווידאו לאפליקציה

הנתיב `camera/stream/` מתחבר ל-`stream/` של ה-ESP32-CAM ומעביר את הבייטים לאפליקציה כפי שהם, בלי חיץ של פריים שלם. הממשק משתמש ב-`httpx` במצב סטרימינג ידני (`send(..., stream=True)`) ולא ב-`async with client.stream(...)`. הסיבה: `StreamingResponse` שולח את הסטטוס והכותרות לפני שהוא מושך את הפריים הראשון מהגנרטור, ולכן בדיקת סטטוס שנעשית בתוך הגנרטור מאוחרת מדי — 200 כבר נשלח, והאפליקציה לא הייתה יודעת שהמצלמה אינה נגישה. ללקוח הייעודי `read=None` (ללא מגבלת קריאה), מפני שהפערים בין פריימים ב-MJPEG אינם קבועים, ומגבלת ברירת המחדל של חמש שניות הייתה הורגת את הזרם. כך האפליקציה אינה פונה ישירות ללוח, והזרם עובד מכל מקום שהבקנד נגיש בו, כולל דרך Tailscale.

```python
@app.get("/camera/stream", dependencies=[Depends(require_app)])
async def proxy_camera_stream():
    """  ...  """
    if not CAMERA_STREAM_URL:
        raise HTTPException(status_code=503, detail="Camera stream not configured")

    headers = {"X-Device-Key": state.device_key}
    try:
        request = _cam_stream_client.build_request("GET", CAMERA_STREAM_URL,
headers=headers)
        upstream = await _cam_stream_client.send(request, stream=True)
    except httpx.ConnectError as exc:
        raise HTTPException(status_code=502, detail=f"Camera unreachable: {exc}") from
exc
    except httpx.TimeoutException as exc:
        raise HTTPException(status_code=504, detail=f"Camera timed out: {exc}") from
exc

    if upstream.status_code != 200:
        await upstream.aclose()
        raise HTTPException(status_code=502, detail=f"Camera returned HTTP
{upstream.status_code}")

    # ...
    content_type = upstream.headers.get(
        "content-type", "multipart/x-mixed-replace; boundary=frame"
    )

    async def relay():
        try:
            async for chunk in upstream.aiter_raw():
                yield chunk
        except httpx.StreamError:
            # ...
            return

    return StreamingResponse(
        relay(), media_type=content_type, background=BackgroundTask(upstream.aclose)
    )
```

קוד 12: העברת זרם MJPEG **לאפליקציה**

4.4.6 שידור בזמן אמת והתראות

WebSocket. `/ws` מקבל את האסימון ב-`?token=` (ל-WebSocket אין דרך ניידת להוסיף כותרות), ונסגר בקוד 1008 **לפני** `accept()` אם האסימון אינו תקף, כך שלקוח לא מאומת אינו נרשם לשידורים ואינו מקבל אפילו הודעה אחת. אחרי קבלת חיבור נשלח מצב מלא מיד, כדי שהאפליקציה לא תחכה לשינוי הבא.

התראות `notifier.notify()` **.ntfy** אינה קורוטינה במכוון: היא רק מתזמנת משימה (`asyncio.create_task`), ומשאירה לה הפניה חזקה בקבוצה `_pending`, מפני ש-asyncio שומר רק הפניה חלשה למשימה פעילה והיא עלולה להימחק באמצע השליחה בלי שום שגיאה. אילו הוגדרה עם `await` בתוך `trigger_alert`, שרת ntfy איטי או לא נגיש היה עוצר את שידור ה-

WebSocket לכל האפליקציות עד חמש שניות (`_TIMEOUT`). שם ה-Topic הוא הסוד היחיד שמגן על ההתראות, ולכן אינו נרשם ביומן.

```
 80 def notify(
 81     server: str,
 82     topic: str,
 83     title: str,
 84     message: str,
 85     priority: str = "high",
 86     tags: str = "rotating_light",
 87 ) -> None:
 88     """  ...  """
 99     if not topic:
100         return
101     # ...
112     task = asyncio.create_task(_send(server, topic, title, message, priority, tags))
113     _pending.add(task)
114     task.add_done_callback(_pending.discard)
```

קוד 13: שליחת התראה ללא חסימה

4.4.7 כלי בדיקה בבקנד

הבקנד נבדק בשתי חבילות משלימות:

- **`pytest`** — בדיקות פנימיות מול אפליקציית ה-ASGI דרך `httpx.ASGITransport`, ללא שרת, ללא פורטים וללא המתנות אמיתיות (פקיעת נעילה ו-Heartbeat מיושן נבדקים על ידי הזזת חותמות זמן). כל בדיקה מקבלת מצב נקי. מספר הבדיקות הנוכחי: 258.
- **`simulate.py`** — 98 בדיקות מול שרת `uvicorn` אמיתי דרך סוקט אמיתי, כולל המתנה של 30 שניות לפקיעת נעילה ועוד 35 שניות לפקיעת Heartbeat. איטית, אך בודקת גם את שרת ה-ASGI ואת ה-WebSocket.

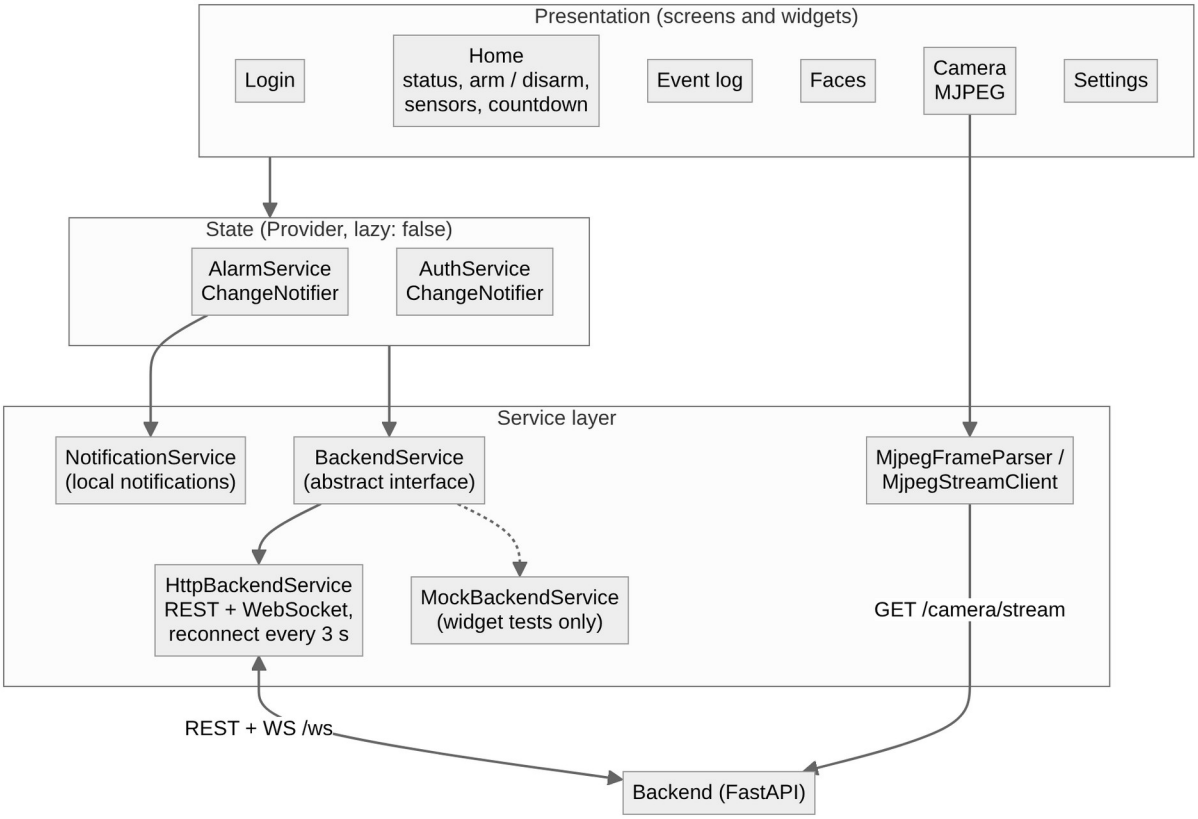
בנוסף, `ruff` מבצע בדיקת סגנון, והכול רץ אוטומטית ב-GitHub Actions בכל Push. פירוט התוצאות בפרק 5.

4.5 מימוש האפליקציה (Flutter, אנדרואיד)

4.5.1 ארכיטקטורת האפליקציה

האפליקציה מפוצלת לשלוש שכבות:

- **שכבת שירות (`BackendService`):** ממשק מופשט בצד Dart. המימוש `HttpBackendService` מדבר עם הבקנד האמיתי (REST + WebSocket), ו-`MockBackendService` נשאר רק לבדיקות וידג'טים ולעבודה בלי שרת.
- **שכבת מצב (Provider [23]):** `AlarmService` ו-`AuthService` מאזינים לזרמי הנתונים של השירות ומעדכנים את הממשק. שניהם מוגדרים `lazy: false` מפני שהם נרשמים לזרמי שידור (Broadcast) בבנאי, והזרמים האלה אינם משדרים מחדש אירועים שכבר קרו. בלי `lazy: false` אירועים ראשונים (כמו התחברות ראשונה) אובדים, וזה הסיבה לבאג "דברים מופיעים רק אחרי דריכה ראשונה".
- **שכבת תצוגה:** מסכים ווידג'טים (Material 3).



איור 21: ארכיטקטורת האפליקציה

4.5.2 תקשורת מול הבקנד

WebSocket עם התחברות חוזרת. `_connect()` פותח חיבור ל-`/ws?token=`, ובכל ניתוק (`onDone` / `onError`) `_scheduleReconnect()` מפעיל ניסיון חוזר כל 3 שניות. באותו רגע הקישור מסומן כמנותק (`_setLinked(false)`), ומסך הבית מציג זאת במקום להישאר קפוא וירוק. הבקשות מבוצעות על ידי `_get/_post`, שמחזירות אובייקט עם הדגל `reachable`: בגרסה קודמת כל שגיאה נבלעה והוחזר `{}`, כך ש"הבקנד לא נגיש" נראה זהה ל"הבקנד אמר לא". במערכת אזעקה זה קונקרטי — לחיצה על DISARM ללא נתיב לבקנד נראתה כנטרול מוצלח. כשלים מגיעים כעת ל-`AlarmService.lastError` ומוצגים כ-SnackBar.

כתובת הבקנד. `BackendConfig` שומר את הכתובת ב-`SharedPreferences`, נטען לפני `runApp`, ונערך במסך ההגדרות; שינוי דורש הפעלה מחדש של האפליקציה (`baseUrl` הוא `final` בכוונה). שלוש דרכי חיבור נבדקו: כתובת LAN (נכשלה ברשת המשתמש, כנראה בגלל בידוד לקוחות בראוטר), `adb reverse` מעל USB (עבדה, אך דורשת כבל ומכשיר אחד) ו-Tailscale (נבחרה לפריסה).

```
157   void _connect() {
158     if (_disposed) return;
159     final token = _token;
160     if (token == null) return;
161
162     // Only the scheme is rewritten, so a token or path containing "http"
163     // can't be mangled the way a blanket replaceFirst on the whole URL could.
164     final wsUrl = baseUrl.startsWith('https')
165         ? baseUrl.replaceFirst('https', 'wss')
166         : baseUrl.replaceFirst('http', 'ws');
167     try {
168       _channel = WebSocketChannel.connect(
169         Uri.parse('$wsUrl/ws?token=${Uri.encodeQueryComponent(token)}'),
170       );
171       _wsSub = _channel!.stream.listen(
172         _handleMessage,
173         onDone: _scheduleReconnect,
174         onError: (_) => _scheduleReconnect(),
175         cancelOnError: true,
176       );
177     } catch (_) {
178       _scheduleReconnect();
179     }
  ⋮
199   void _scheduleReconnect() {
200     _setLinked(false);
201     if (_disposed || _token == null) return;
202     _reconnectTimer?.cancel();
203     _reconnectTimer = Timer(const Duration(seconds: 3), _connect);
204   }
205
```

קוד 14: חיבור WebSocket **עם התחברות חוזרת**

4.5.3 פענוח זרם MJPEG

ל-Flutter אין תמיכה מובנית ב-MJPEG, וחבילה מתוחזקת שתומכת גם בכותרות אימות לא נמצאה, ולכן נכתב `MjpegFrameParser` שמפרסר ידנית את ה-`multipart/x-mixed-replace`. הפריימים מגיעים בחתיכות רשת שאינן קשורות לגבולות הפריימים, ולכן `addChunk` מצבר חתיכות ומחזיר את כל הפריימים שהושלמו, תוך השארת פריים חלקי במאגר. הפירוש נעשה לפי `Content-Length`. לקוח הזרם (`MjpegStreamClient`) מתחבר מחדש אוטומטית בכל נפילה, והמאגר מוגבל ל-8 MB כדי שזרם פגום לא ידלוף זיכרון. התצוגה משתמשת ב-`ValueNotifier` וב-`RepaintBoundary` ולא ב-`setState`, כדי שכל כרטיס המצלמה (תג, כיתוב, מסגרת) לא יבנה מחדש 30 פעמים בשנייה.

```
  List<Uint8List> addChunk(List<int> chunk) {
    _buffer.addAll(chunk);
    final frames = <Uint8List>[];

    while (true) {
      final boundaryStart = _indexOf(_buffer, _boundary, 0);
      if (boundaryStart == -1) {
        if (_buffer.length > _maxBufferBytes) _buffer.clear();
        break;
      }

      final headerStart = boundaryStart + _boundary.length;
      final headerEnd = _indexOfCrlfCrlf(_buffer, headerStart);
      if (headerEnd == -1) break; // headers haven't fully arrived yet

      final headerText = ascii.decode(_buffer.sublist(headerStart, headerEnd),
allowInvalid: true);
      final contentLength = _parseContentLength(headerText);
      final bodyStart = headerEnd + 4; // past \r\n\r\n

      if (contentLength == null) {
        // ...
        _buffer.removeRange(0, bodyStart);
        continue;
      }

      final bodyEnd = bodyStart + contentLength;
      if (_buffer.length < bodyEnd) break; // frame not fully arrived yet

      frames.add(Uint8List.fromList(_buffer.sublist(bodyStart, bodyEnd)));
      _buffer.removeRange(0, bodyEnd);
    }
    return frames;
  }
```

קוד 15: פענוח זרם MJPEG

4.5.4 ממשק המשתמש (GUI)

העיצוב נבנה כך שיראה כלוח בקרה הנדסי ולא כאפליקציית צרכנים גנרית: ערכת Material 3 כהה בגוון ירקרק-אפור (כמו מארז PCB כהה), עם צבעים הנושאים משמעות — **ירוק** (`#43C285`) לנטרול, **אדום** (`#E14B3C`) לדרוך או אזעקה, **כתום** (`#E2953A`) לדריכה בתהליך, **כחול** (`#4198D1`) לאלמנטים אינטראקטיביים. הגופנים (IBM Plex Sans/Mono, Big Shoulders Display) מצורפים כנכסים באפליקציה ולא נטענים מהרשת, מפני שמערכת אזעקה חייבת להיות קריאה גם ללא אינטרנט, ובדרכי הגישה מרחוק המכשיר מגיע לבקנד לעיתים ללא נתיב כללי לאינטרנט.

המסכים: **Login** (עם הצגת נעילה אחרי 3 ניסיונות שגויים — הבקנד הוא הסמכות היחידה לנעילה, והאפליקציה רק משקפת אותה); **Home** (אינדיקטור מצב, בחירת Home/Away, פאנל חיישנים לפי חיישן וסטטוס כל לוח, וספירה לאחור של יציאה וכניסה); **Event Log** (פילטר לפי חומרה); **Faces** (רישום פנים ונטרול לפי פנים); **Camera** (זרם חי); **Settings** (שינוי סיסמה, Night Mode, איש קשר לחירום, ntfy וכתובת הבקנד).

[הכנס תמונה כאן: צילומי מסך של מסך Login — מצב רגיל, ומצב נעילה לאחר ניסיונות שגויים (הצגת ספירה לאחור)]

איור 22: מסך ההתחברות

[הכנס תמונה כאן: צילומי מסך של מסך Home בארבעת המצבים — מנוטרל (ירוק), בדריכה (כתום, ספירה לאחור), דרוך Home/Away (אדום), נטרול כניסה (Entry Delay עם ספירה), ואזעקה (Triggered עם סימון החיישן שדיווח)]

איור 23: מסך הבית במצבי המערכת

[הכנס תמונה כאן: צילומי מסך של מסכי Event Log (עם פילטר), Faces (רשימת פנים מאושרות והגדרות נטרול), Camera (זרם חי) ו-Settings]

איור 24: מסכי יומן האירועים, הפנים, המצלמה וההגדרות

[הכנס תמונה כאן: צילום מסך של התראת ntfy בטלפון כשהאפליקציה סגורה, ושל התראת ההסלמה הדחופה]

איור 25: התראות Push

4.6 אבטחת מידע — סיכום ההגנות

טבלה 8: איומים והגנות

איום	הגנה	מימוש
זיוף דיווח פריצה או "נוטרל" ברשת	מפתח מכשיר משותף בכל בקשה מלוח	`X-Device-Key`, `require_device`; 401 בלעדיו
נטרול מרחוק ללא הרשאה	אסימון סשן אקראי, בתוקף 12 שעות	`secrets.token_urlsafe(32)`; שינוי סיסמה מבטל את שאר הסשנים
גניבת מסד הסיסמאות	גיבוב איטי עם Salt	scrypt ($N=2^{14}$, $r=8$, $p=1$, Salt 16 בתים), כ-16 MiB זיכרון לגיבוב [13]
ניחוש סיסמה	נעילה לאחר 3 ניסיונות ל-30 שניות; השוואה בזמן קבוע	`compare_digest`; הבקנד הוא הסמכות
פקודת נטרול ישירות ללוח הדלת	אימות מפתח גם בכיוון ההפוך	`authorized()` בכל handler
WebSocket ללא אימות	סגירה לפני `accept()`	קוד 1008

איום	הגנה	מימוש
route חדש שנשכח להגן עליו	בדיקה פרמטרית על טבלת כל ה-routes	`tests/test_auth.py`
דליפת סודות לקוד	`secrets.h` מחוץ לגיט	`.gitignore` + תבנית
חשיפה לאינטרנט הפתוח	רשת פרטית עם חיבור יוצא בלבד	Tailscale (WireGuard) [14][15]
זיוף נטרול באמצעות תמונה	שתי בדיקות חיות; נטרול פנים כבוי כברירת מחדל	סעיף 4.4.4

מגבלות והחלטות מכוונות. (א) סיסמת האפליקציה וקוד הקודן הם **סוד משותף אחד**, שחייב להיות 4–8 ספרות, כך שמרחב החיפוש קטן מ-$\sum_{k=4}^{8} 10^k = 111{,}110{,}000 \approx 1.1 \times 10^8$ צירופים. זו החלטה מושכלת למערכת ביתית, והיא ראויה לבחינה מחדש בפריסה בסביבה פחות מהימנה. (ב) מפתח המכשיר המשותף שמאפשר דיווח פריצה הוא גם המפתח שצופה בזרם הווידאו. (ג) הנעילה חלה על ההתחברות והחלפת הסיסמה בלבד; הקודן הפיזי מאפס את המאגר לאחר הקשה שגויה, ללא נעילה. (ד) תיעוד ה-API (`/docs`) פתוח ולכן סכמת ה-API ציבורית; ניתן לכבותו בפריסה אמיתית. (ה) סיסמת ברירת המחדל בהרצה ראשונה (`homeguard`) חייבת להישתנות בהתקנה.

4.7 כלים, תוכנות וסביבות פיתוח

תת-פרק זה מרכז את כל התוכנות ששימשו בפרויקט, בחלוקה לפי תפקיד.

טבלה 9: תוכנות וסביבות הפיתוח

קטגוריה	כלי / סביבה	שימוש בפרויקט
קושחה (IDE)	Arduino IDE, `arduino-cli` + ליבת `esp32:esp32` (FQBN: `esp32`) ו-`esp32cam`	כתיבה, קומפילציה וצריבה של קושחת הלוחות; Serial Monitor לאבחון
בקנד (סביבה/IDE)	Python 3 עם `venv`; FastAPI, Uvicorn, Pydantic, httpx, OpenCV (contrib), NumPy	הבקנד וזיהוי הפנים
בדיקות ואיכות	pytest, Ruff, `simulate.py`, GitHub Actions	בדיקות אוטומטיות ו-CI
אפליקציה (סביבה/IDE)	Flutter / Dart, Android SDK, JDK 17, Gradle	פיתוח ובניית APK (הבנייה מתבצעת דרך WSL2, בשל תקלת Winsock ב-Windows)
סביבת עבודה	Windows + WSL2 (Ubuntu), PowerShell, `adb`	בנייה, התקנה וניפוי על מכשיר פיזי; `adb reverse` לחיבור USB
בקרת גרסאות	Git, GitHub	ניהול הקוד והיסטוריית השינויים

קטגוריה	כלי / סביבה	שימוש בפרויקט
רשת והתראות	Tailscale, ntfy	גישה מרחוק מאובטחת והתראות Push
עיצוב GUI	*[להשלים: Figma / ידני בקוד Flutter (Material 3)]*	תכנון הממשק
שרטוטי חשמל	*[להשלים: EasyEDA / KiCad / Fritzing / ידני]*	סכמות החיבורים (איורים 10–13)
תרשימי זרימה ומצבים	*[להשלים: draw.io / Lucidchart / Mermaid / PowerPoint]*	תרשימי הבלוקים, הזרימה והרצף
מידול תלת-ממדי	*[להשלים: Tinkercad / Fusion 360 / SolidWorks — או "לא נעשה שימוש"]*	מארזים ותושבות (איור 14)
הדפסה תלת-ממדית	*[להשלים: תוכנת Slicer (Cura / PrusaSlicer) ומדפסת — או "לא נעשה שימוש"]*	הדפסת חלקים
כתיבת הדוח	*[להשלים: Google Docs / Word]*	הפקת הדוח
כלים מבוססי בינה מלאכותית	*[להשלים/לאשר לפי הנחיות הקורס: Claude Code]*	סיוע בפיתוח וכתיבה

הספריות העיקריות בבקנד: FastAPI [8], Uvicorn [26], Pydantic [27], httpx [22], OpenCV [29] ו-NumPy [28], כאשר מודלי YuNet ו-SFace לקוחים מ-OpenCV Zoo [30]. באפליקציה: Provider [23], web_socket_channel [34], http [38], shared_preferences [35], flutter_local_notifications [36] ו-google_fonts [37], על בסיס Material Design 3 [39]. הבדיקות והאינטגרציה הרציפה נשענות על pytest [31], GitHub Actions [32] ו-arduino-cli [33].

(הרשימה המלאה והמאוחדת של כל המקורות מופיעה בפרק 7 בקובץ `03_tests_summary_bom_refs.md`.)

5. בדיקות, ניסויים ותוצאות

פרק זה מתאר כיצד נבדקה המערכת ומה התקבל. הבדיקות חולקו לארבע רמות, מהזולה והמהירה ועד לבדיקה על חומרה אמיתית, וכל רמה תופסת סוג שונה של תקלות.

5.1 אסטרטגיית הבדיקות

טבלה 10: רמות הבדיקה

רמה	מה נבדק	כלי	אופן הרצה
1. בדיקות פנימיות (In-process)	לוגיקת הבקנד: מכונת מצבים, אימות, נעילה, זיהוי חיות, התראות, שמירה	`pytest` מול אפליקציית ה-ASGI דרך `httpx.ASGITransport` [31]	אוטומטית בכל Push
2. בדיקות מקצה לקצה	הבקנד כשרת אמיתי: ASGI, WebSocket, זמנים אמיתיים	`simulate.py` מול `uvicorn` חי	אוטומטית ב-CI ובהרצה ידנית
3. קומפילציית קושחה	ארבעת ה-sketches מתקמפלים (כולל FQBN נפרד למצלמה)	`arduino-cli` [33]	אוטומטית ב-CI
4. בדיקות חומרה ומערכת	התנהגות על לוחות, חיישנים, קודן ומצלמה אמיתיים, מול האפליקציה	ניסוי ידני, Serial Monitor	ידנית במעבדה

ההפרדה בין שתי הרמות הראשונות מכוונת: `pytest` אינו מחכה ואינו פותח פורטים (פקיעת נעילה ו-Heartbeat מיושן נבדקים על ידי הזזת חותמות זמן), ולכן מאפשר משוב מהיר תוך כדי עבודה. `simulate.py` איטי (כולל המתנה אמיתית של 30 שניות לפקיעת נעילה ועוד 35 שניות לפקיעת Heartbeat), אך בודק את מה ש-`pytest` אינו יכול: את שרת ה-ASGI ואת תעבורת ה-WebSocket.

5.2 בדיקות אוטומטיות של הבקנד

5.2.1 חבילת `pytest`

הבדיקות הורצו מחדש לצורך הדוח, בסביבה נקייה: Linux, Python 3.13, עם גרסאות התלויות המקובעות ב-`requirements.txt`. **התוצאה: 258 בדיקות עברו, 0 נכשלו, בזמן של כ-90 שניות (89 שניות בהרצה המצולמת באיור 26), עם אזהרת הוצאה משימוש (Deprecation) אחת מצד ספריית Starlette.** כל בדיקה מקבלת מצב נקי באמצעות טעינה מחדש של מודול המצב לתיקייה זמנית.

טבלה 11: פירוט חבילת ה-pytest לפי קובץ

קובץ בדיקות	מס' בדיקות	תחום הכיסוי
`test_auth.py`	72	כיסוי כל ה-routes, דחיית קריאות אנונימיות ואסימון מזויף, מפתח מכשיר, נעילה לאחר 3 ניסיונות ופקיעתה, גיבוב scrypt עם Salt, פקיעת אסימונים, אי-חסימת ה-Event Loop
`test_face_disarm_and_schedule.py`	40	נטרול לפי פנים (כבוי כברירת מחדל, חיות), ספירת Entry Delay וה-Watchdog, Night Mode, הסלמה, הגבלת קצב יומן
`test_alarm_flow.py`	27	מעברי מצבים, עקיפת חיישן ב-Home, דיווחי בקר הדלת, מרוץ בין arm ל-disarm, ניסיון חוזר לפקודת הזמזם
`test_boards_and_persistence.py`	27	Heartbeat ופקיעתו, זיהוי לוח לא מוכר, שמירה ושחזור מהדיסק, קובץ מצב פגום, כיבוי נקי
`test_faces_and_api.py`	23	בחירת מנוע זיהוי (SFace / LBPH), תמונות פגומות (422 במקום 500), CRUD פנים, צורת ה-wire מול Flutter, WebSocket
`test_shared_credential.py`	21	הסיסמה המשותפת לקודן, סנכרון `secrets.h`, אי-דליפה ליומן ולשידור
`test_liveness.py`	19	לוגיקת זיהוי החיות (תנועה ומודל), גבולות הספים, הכשל לכיוון הבטוח, חישוב חתך הפנים
`test_ip_check.py`	15	אזהרה כשה-IP הצרוב בלוח אינו תואם ל-IP של המחשב
`test_notifications.py`	14	התראות ntfy: אי-חסימה, אי-דחיפה לאירוע שהתעלמו ממנו, אי-רישום ה-Topic ביומן
סה"כ	**258**	

```
$ cd python && pytest -q
.....................................................................  [ 27%]
.....................................................................  [ 55%]
.....................................................................  [ 83%]
.....................................                                  [100%]
============================== warnings summary ===============================
tests/test_faces_and_api.py::test_websocket_requires_a_token
  .../venv/lib/python3.13/site-packages/fastapi/testclient.py:1: StarletteDeprecationWarning: Using `httpx` with `starlette.testclient` is deprecated; install `httpx2` instead.
    from starlette.testclient import TestClient as TestClient  # noqa

-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
258 passed, 1 warning in 89.19s (0:01:29)
```

איור 26: תוצאת הרצת חבילת ה-pytest

מכלול הבדיקות נכתב בעיקר סביב באגים אמיתיים שנתגלו בביקורת הקוד ובהרצה על חומרה, ולא סביב כיסוי שורות כללי. שמות הבדיקות משקפים זאת, למשל `test_a_disarm_during_the_exit_delay_is_not_undone_three_seconds_later`, `test_the_controller_cannot_arm_out_of_a_live_alarm`, ו-`test_one_typo_after_a_lockout_expires_does_not_re_lock_and_re_alarm`

`test_the_entry_delay_alarms_even_if_the_controller_goes_silent`. כך כל באג שתוקן נשאר נעול מפני חזרה.

5.2.2 הסימולציה מקצה לקצה (`simulate.py`)

הסימולציה הורצה מול שרת `uvicorn` אמיתי על סוקט אמיתי, עם מפתח מכשיר ידוע וקובץ מצב נקי. **התוצאה: 98 בדיקות עברו, 0 נכשלו.**

טבלה 12: פירוט הסימולציה לפי נושא

נושא	בדיקות שעברו
בריאות ואכיפת אימות	8
התחברות ונעילה (כולל המתנה אמיתית של 30 שניות)	23
דריכה ונטרול	4
עקיפת חיישן במצב Home	3
שמירת העקיפה לאחר דיווח הבקר	3
התראות לפי חיישן (שלושת החיישנים, מצב Away)	9
ניהול פנים (CRUD)	6
זיהוי פנים ומצלמה	8
הגדרות	4
Entry Delay	5
נטרול לפי פנים	5
התראות ו-Night Mode	8
נתיבי ה-ESP32	10
פקיעת Heartbeat של לוח (המתנה אמיתית של 35 שניות)	2
סה"כ	**98**

```
$ cd python && python simulate.py (last 32 lines of the output)
  PASS  the disarm is attributed in the log

=== Notifications & night-mode schedule ===
  PASS  ntfy settings accepted
  PASS  ntfy settings persisted
  PASS  the topic is not leaked into the event log
  PASS  test notification reports configured
  PASS  test notification reports not-configured when off
  PASS  night-mode hours accepted
  PASS  night-mode hours persisted
  PASS  out-of-range hours are clamped, not rejected

=== ESP32 endpoints ===
  PASS  heartbeat accepted with device key
  PASS  esp32_online becomes true after heartbeat
  PASS  state reports all three boards
  PASS  heartbeating board is online
  PASS  boards that never reported are offline
  PASS  esp32 sensor-event accepted
  PASS  esp32 sensor-event triggers alarm
  PASS  esp32 state-change accepted
  PASS  esp32-reported disarm reflected in state
  PASS  esp32-reported arm reflected in state (no exit delay — controller already did its own)

=== Board liveness ages out (takes ~35s) ===
  PASS  vibration board online right after its heartbeat
  (waiting 35s for the heartbeat to go stale...)
  PASS  vibration board goes offline once heartbeats stop

================================================
98 passed, 0 failed
================================================
```

איור 27: תוצאת הסימולציה מקצה לקצה

5.2.3 איכות קוד ואינטגרציה רציפה

`ruff check` עבר ללא ממצאים. הכול רץ אוטומטית ב-GitHub Actions בארבע עבודות (Jobs) בכל Push וכל Pull Request: **(1)** lint ו-**(2)** `pytest`; בקנד — הסימולציה החיה; **(3)** קושחה — קומפילציה של כל ארבעת ה-sketches (שלושה עם `esp32:esp32:esp32` והמצלמה עם `esp32:esp32:esp32cam`, כי בנייה שגויה של המצלמה מייצרת קושחה שאינה מסוגלת להפעיל אותה); **(4)** אפליקציה — `flutter analyze` ו-`flutter test`. באפליקציה קיים קובץ בדיקות וידג'טים אחד (`widget_test.dart`, 14 מקרי בדיקה), שרץ מול `MockBackendService` מפני שאין שרת אמיתי בזמן `flutter test`.

[הכנס צילום מסך של ריצה ירוקה של כל ארבע העבודות ב-GitHub Actions (לשונית Actions במאגר)]

איור 28: תוצאת האינטגרציה הרציפה

[הכנס כאן את פלט `flutter analyze` ו-`flutter test` מהריצה האחרונה — לא הורצו בעת כתיבת הדוח]

5.3 בדיקות חומרה ומערכת

המערכת הורכבה ונבדקה על חומרה אמיתית בכל ארבעת הלוחות (ולא רק בקומפילציה): החל מחיבור WiFi ודיווח לבקנד, דרך הקודן והדלת, ועד למערכת המשולבת מול האפליקציה. חלק מהבדיקות בוצעו בנתיבי הגישה הנבדלים (`adb reverse` מעל USB ו-Tailscale) כדי לוודא שהאפליקציה מגיעה לבקנד גם מחוץ לרשת המקומית. הקודן נבדק תחילה באמצעות Serial Monitor (מצב סימולציה שנשאר כגיבוי) ורק אחר כך בחיבור פיזי.

טבלה 13: בדיקות המערכת על חומרה אמיתית

מס'	תיאור הבדיקה	תוצאה צפויה	סטטוס
1	הפעלת ארבעת הלוחות	כל לוח מתחבר ל-WiFi ומדפיס את ה-IP שקיבל	עבר
2	חציית שדה החיישן האולטרה-סוני במערכת דרוכה	דיווח חדירה לבקנד, מעבר ל-Triggered, התראה באפליקציה	*[לאמת]* (הפרמטרים כוונו לאחר הבדיקה הראשונה, ראה 5.3.1)
3	מכה בחיישן הרעידות (מערכת דרוכה ב-Away)	דיווח, Triggered; ב-Home — התעלמות	*[לאמת]*
4	הקשת קוד נכון + # בחלון הנטרול	נטרול, והזמזמים נכבים ונשארים כבויים	עבר (לאחר תיקון, ראה 5.3.1)
5	פתיחת דלת על מערכת דרוכה והמתנה 25 שניות	צפצופים מואצים, ואז אזעקה	*[לאמת]*
6	חדירה שדווחה מלוח אחר (אולטרה-סוני / רעידות)	הזמזמים בבקר הדלת מופעלים (`/trigger-alarm`)	*[לאמת]* (הקוד תוקן, ראה 5.3.1)
7	דריכה ונטרול מהאפליקציה	הפקודה מגיעה לבקר הדלת (`hardware: true`) והמצבים מסונכרנים	*[לאמת]*
8	ניתוק לוח מהחשמל	הלוח מסומן כלא מחובר באפליקציה תוך כ-35 שניות לכל היותר	*[לאמת]*

מס'	תיאור הבדיקה	תוצאה צפויה	סטטוס
9	זרם וידאו חי מה--ESP32 CAM לאפליקציה	זרם רציף עם אימות	עבר (21 fps ב-VGA דרך Tailscale)
10	זיהוי פנים מאושרות בפריימים מה-ESP32-CAM	התאמה, רישום ביומן, ולפי הגדרה נטרול	*[לאמת]*
11	התראת Push כשהאפליקציה סגורה	התראת ntfy מגיעה לטלפון	*[לאמת]*

הערה לגבי הסטטוס: "עבר" נרשם רק כשהתוצאה מתועדת בהיסטוריית הפרויקט. שורות המסומנות "[לאמת]" מתארות בדיקות שבוצעו או מתוכננות, אך תוצאתן תושלם מיומן המעבדה.

[הכנס תמונה כאן: צילומי הבדיקות על החומרה — (א) צילום מסך של Serial Monitor בקר הדלת בזמן ALARM → ENTRY_DELAY → נטרול; (ב) צילום של האפליקציה במצב Triggered עם סימון החיישן; (ג) צילום של הטלפון עם התראת ntfy כשהאפליקציה סגורה]

איור 29: תיעוד בדיקות החומרה

[הכנס כאן קישור לסרטון הבדיקה (למשל חדירה ← ספירה ← אזעקה ← נטרול בקודן)]

5.3.1 ממצאי העבודה על החומרה

הרצת המערכת על חומרה אמיתית חשפה ארבע תקלות שהבדיקות הפנימיות לא יכלו לתפוס, ותוקנו בעקבותיה:

1. **הזמזמים נשארו דלוקים לאחר הקשת קוד נכון.** בענף הנטרול של מצב ENTRY_DELAY חסרה פקודת `break`, ולכן הביצוע נפל ישירות ללוגיקת הצפצוף המואץ באותה איטרציה של `loop()` והדליק את הזמזמים מיד אחרי שכבו. התקלה שוחזרה על השולחן, תוקנה ואומתה.
2. **חדירה בחיישן שאינו הדלת לא הפעילה זמזם.** הזמזמים בבקר הדלת הגיבו רק לחיישן הדלת ולקודן שלו. בעקבות זאת נוסף הנתיב `POST /trigger-alarm`, והבקנד קורא לו לכל מקור למעט הדלת.
3. **כיוונון החיישן האולטרה-סוני.** הערכים הרפויים שנבחרו תחילה (8 חריגות רצופות, סבילות של 30 ס"מ, דגימה ב-10 הרץ) איפשרו לחפצים אמיתיים לחצות ללא זיהוי. הערכים הוחלפו ב-5 חריגות רצופות, סבילות של 10 ס"מ ודגימה בכ-15 הרץ, והבסיס כויל אוטומטית מ-10 הקריאות הראשונות בעת האתחול במקום ערך קבוע של 200 ס"מ.
4. **חסימת הבקר בזמן דיווח.** דיווח סינכרוני חסם את `loop()` ואיתו את הטיפול בפקודות נכנסות; הפתרון (תור FreeRTOS ו-Task ייעודי) מתואר בסעיף 4.3.4. מספר פתרונות ביניים (הגבלת Timeout לשנייה) קיצרו את החלון אך לא סגרו אותו.

5.4 ביצועים וניסויים

טבלה 14: מדדי ביצועים ופרמטרים שנמדדו או נגזרו

מדד	ערך	מקור
זמן ריצה של חבילת pytest (258 בדיקות)	כ-90 שניות (84–89 בשתי הרצות)	נמדד לצורך הדוח (Linux, Python 3.13)
קצב זרם וידאו מקצה לקצה (VGA, דרך Tailscale)	כ-21 פריימים לשנייה	נמדד בהרצה על חומרה (מתועד בהיסטוריית הפרויקט)
זמן עיבוד זיהוי פנים לפריים	כ-21 ms	נמדד בפיתוח (מתועד בהיסטוריית הפרויקט)
זמן אימון מחדש של המזהה (50 תמונות)	כ-556 ms	נמדד בפיתוח (מתועד בהיסטוריית הפרויקט)
גיבוב סיסמה (scrypt)	כ-50 ms לגיבוב	נמדד בפיתוח (מתועד בהיסטוריית הפרויקט)
קצב דגימת החיישן האולטרה-סוני	כ-15 הרץ (67 ms)	הגדרת קושחה
זמן אימות חדירה מינימלי (אולטרה-סוני)	$\geq 5 \times 67\,ms \approx 0.34\,s$	חישוב מהגדרות הקושחה
קצב שליחת פריימים לזיהוי פנים	2 פריימים לשנייה	הגדרת קושחה (`CAPTURE_INTERVAL_MS = 500`)
משך חלון הנטרול (Entry Delay)	25 שניות	הגדרת קושחה
זמן אזעקה מרבי כשבקר הדלת שותק	$25+5=30$ שניות	חישוב: משך הנטרול + `ENTRY_DELAY_GRACE`
זמן מרבי לזיהוי לוח שהתנתק	$30+5=35$ שניות	חישוב: `BOARD_TIMEOUT` + מחזור הניטור
זמן חסימה מרבי של בקשה יוצאת מהלוח	כ-2 שניות	חישוב: Timeout של שנייה לחיבור ושנייה לתגובה
סיכוי לזיהוי צרור רעידות בן 2 ms	כ-4% בדגימה כל 50 ms, לעומת תפיסת כל קצה עולה בפסיקה	ניתוח תכנוני (סעיף 4.3.3)

הערכים מהיסטוריית הפרויקט נמדדו על מכונת הפיתוח, ויש לאשר אותם או למדוד מחדש לפני ההגשה. הערכים הנגזרים הם גבולות תכנוניים ואינם מדידות.

להלן שלושה ניסויים שיש להשלים את נתוניהם מיומן המעבדה. הנתונים אינם שמורים במאגר הקוד, ולכן לא מולאו.

ניסוי א' — אפיון החיישן האולטרה-סוני. מטרתו לבדוק באילו מרחקים וגדלי אובייקט החיישן מזהה חדירה, וכמה אזעקות שווא מתקבלות במצב סרק.

[הכנס טבלה/גרף כאן: מרחק האובייקט מהחיישן (ס"מ) מול התוצאה (זוהה / לא זוהה) ומספר הקריאות החריגות הרצופות; ולצידם מספר אזעקות השווא במצב סרק במשך פרק זמן נתון (למשל 24 שעות)]

איור 30: אפיון החיישן האולטרה-סוני

ניסוי ב' — דיוק זיהוי הפנים ועמידות בפני זיוף. מטרתו למדוד התאמות נכונות, החמצות והתאמות שגויות, ולבחון ניסיונות זיוף באמצעות תמונה מודפסת ומסך טלפון.

טבלה 15: תוצאות זיהוי פנים וזיהוי חיות

תרחיש	מספר ניסיונות	זוהה נכון	הוחמץ	זוהה בטעות	עבר בדיקת חיות
אדם מאושר, פנים חיות	*[להשלים]*	*[להשלים]*	*[להשלים]*	—	*[להשלים]*
אדם לא מאושר	*[להשלים]*	—	—	*[להשלים]*	—
תמונה מודפסת של אדם מאושר	*[להשלים]*	—	—	—	*[להשלים]*
מסך טלפון עם תמונה/וידאו של אדם מאושר	*[להשלים]*	—	—	—	*[להשלים]*

ניסוי ג' — השהיה מאירוע להתראה. מטרתו למדוד את הזמן מרגע דיווח חיישן ועד להתראת ה-WebSocket באפליקציה ועד להתראת ה-Push של ntfy.

[הכנס טבלה כאן: מספר מדידות של השהיה (ms) לפי נתיב — חיישן ← בקנד, בקנד ← אפליקציה (WebSocket), בקנד ← טלפון (ntfy) — עם ממוצע וטווח]

5.5 ניתוח התוצאות

1. הבדיקות האוטומטיות מכסות את הלוגיקה, החומרה מכסה את העולם הפיזי. 258 בדיקות פנימיות ו-98 בדיקות מקצה לקצה עוברות, ולכן לוגיקת מצב האזעקה, האימות והנעילה מתנהגת כמתוכנן. עם זאת, ארבע התקלות שנמצאו על החומרה (סעיף 5.3.1) היו כולן ברמת האינטגרציה בין הקוד למציאות הפיזית: פקודת `break` חסרה ב-`switch`, בקר שאינו מגיב בזמן דיווח, וסף חיישן שאינו מתאים לחפץ אמיתי. אף אחת מהן לא הייתה נתפסת בבדיקה פנימית של הבקנד בלבד.

2. בדיקות שנכתבו סביב כשלים. הקשר בין שמות הבדיקות לבאגים שנמצאו משקף גישה של "חוזים": כל פער בין שני רכיבים שכל אחד מהם נכון בנפרד נקבע בבדיקה. דוגמה מובהקת היא המרוץ בין `arm()` ל-`disarm()` — כל אחד תקין לבדו, והכשל נוצר רק בחלון זמן צר ביניהם.

3. מגבלות הבדיקות. (א) לקושחה אין בדיקות אוטומטיות; היא נבדקת בקומפילציה ב-CI ובהרצה ידנית. מצבי הבקר נבדקו באמצעות הקשות על הקודן ובאמצעות Serial Monitor. (ב) ספי זיהוי החיות (0.008 ו-0.6) הם הערכות שלא כוילו מול מערך זיופים מסודר, ולכן אין להסיק מהמערכת מחויבות להגנה מפני זיוף. (ג) סימולציית ה-`simulate.py` מוגבלת למה שהבקנד חושף: היא אינה בודקת את החומרה. (ד) עומס ארוך טווח (שעות עד ימים) ויציבות WiFi לא נבדקו באופן מסודר, ויש להתייחס לכך כמגבלה.

6. סיכום ומסקנות

6.1 עמידה במטרות הפרויקט

טבלה 16: עמידה בדרישות

דרישה	מקור	סטטוס	הערות
זיהוי חדירה (חיישן היקפי/דלת) או פריצה בכוח	Must	**הושלם**	חיישן אולטרה-סוני במקום לייזר; חיישן דלת מגנטי; שני חיישני רעידות
נטרול באמצעות קודן פיזי	Must	**הושלם**	קודן 3x4; קוד בן 4–8 ספרות הנשמר ב-NVS
אזעקה קולית	Must	**הושלם**	שני זמזמים בבקר הדלת; הפעלה גם מחדירה שדווחה מלוח אחר
בנק פנים מאושרות ונטרול לפי פנים	Nice-to-have	**הושלם**	YuNet + SFace + שתי בדיקות חיות; כבוי כברירת מחדל
דאשבורד יומן אירועים בזמן אמת	Nice-to-have	**הושלם**	WebSocket, פילטר לפי חומרה
שינוי הקוד הסודי מהאפליקציה	Nice-to-have	**הושלם**	קוד משותף לאפליקציה ולקודן, נדחף ל-NVS של הבקר
נעילה חכמה לאחר 3 הקשות שגויות בקודן	תוספת בהצעה	**לא הושלם**	נעילה ממומשת בהתחברות לאפליקציה בלבד; הקודן הפיזי מאפס את המאגר ללא נעילה
שרת Web על ה-ESP32 לשליטה מרחוק	תוספת בהצעה	**הוחלף**	נבנתה אפליקציית Flutter מול בקנד, לצורך התראות ויומן
התראות Push גם כשהאפליקציה סגורה	מעבר להצעה	**הושלם**	ntfy
זרם וידאו חי	מעבר להצעה	**הושלם**	MJPEG מה-ESP32-CAM
Night Mode ו-Entry Delay עם ספירה באפליקציה	מעבר להצעה	**הושלם**	
חיוג אוטומטי לגורם חירום	מעבר להצעה	**לא ממומש**	דורש ספק טלפוניה; נשלחת התראת הסלמה

מסקנה: כל דרישות ה-Must ו-Nice-to-have מומשו, והפרויקט הוסיף מעבר להצעה התראות Push, זרם וידאו חי ו-Night Mode. שני סעיפים מההצעה לא מומשו כלשונם: הנעילה בקודן הפיזי לא מומשה, ושרת ה-Web על ה-ESP32 הוחלף באפליקציה ובבקנד.

6.2 אתגרים והפתרונות להם

טבלה 17: אתגרים מרכזיים

אתגר	אבחון / סיבה	פתרון
המערכת הייתה פתוחה: `POST /disarm` ללא אישורים החזיר 200	אף route לא בדק זהות	אסימוני סשן, מפתח מכשיר ללוחות, scrypt, נעילה, בדיקה פרמטרית על כל ה-routes
דגל "מחובר" שננעל על true — לוח יכול היה למות והאפליקציה הראתה ONLINE	דגל יחיד שאף דבר לא מנקה	Heartbeat לכל לוח, פקיעה לאחר 30 שניות, ושם הלוח באפליקציה
מצב Home הפסיק להתעלם מחיישן הרעידות	בקר הדלת מדווח רק דרוך/מנוטרל, וה-bypass אבד כשהדיווח שלו קבע את המצב	שמירת מצב הדריכה האחרון והפעלת ה-bypass מחדש בעת דיווח הבקר
בקר הדלת חסום בזמן דיווח ואינו מגיב לזמזם	`loop()` קורא `handleClient()` פעם אחת לאיטרציה; בקשה סינכרונית חוסמת עד 10 שניות	Timeout של שנייה, ולבסוף תור FreeRTOS ו-Task ייעודי
הזמזמים נדלקו מחדש אחרי נטרול נכון	חסר `break` בענף הנטרול	הוספת `break` ובדיקה על החומרה
ניתוק החשמל של בקר הדלת בזמן Entry Delay עקף את האזעקה לגמרי	אף אחד לא וידא שהבקר ידווח על התוצאה	Watchdog בבקנד המעלה אזעקה בעצמו לאחר זמן הנטרול ועוד 5 שניות
מרוץ בין `arm()` ל-`disarm()` הותיר בקנד דרוך מעל חומרה מנוטרלת	המשימה נשמרה רק אחרי שידור המצב	בדיקת מצב חוזרת בסיום `_finish_arming`
נעילה שפגה, ושגיאת הקלדה אחת הפעילה שוב נעילה ואזעקה מלאה	מונה הניסיונות לא אופס בפקיעת הנעילה	`_clear_expired_lockout`
האפליקציה הציגה "נטרול הצליח" ללא נתיב לבקנד	כל שגיאת רשת נבלעה והוחזר `{}`	`_Result` עם `reachable`, שגיאות ב-SnackBar, והדגל `hardware`
זיהוי פנים: תמונה מנופפת עברה את בדיקת התנועה	רעד היד יוצר מספיק תזוזה	הוספת מודל MiniFASNetV2 כבדיקה שנייה בלתי תלויה
בניית Android/Gradle נכשלה (`Unable to establish loopback connection`)	תקלת Winsock ברמת המערכת, שלא נפתרה באיפוס ובתיקון המערכת	בנייה דרך WSL2
טלפון פיזי לא הגיע לבקנד ברשת ה-WiFi של האוניברסיטה	בידוד לקוחות בראוטר או חומת אש נוספת	`adb reverse` בפיתוח ו-Tailscale בפריסה, ללא פתיחת פורטים לאינטרנט

אתגר	אבחון / סיבה	פתרון
IP של המחשב משתנה בין הרצות, ולוח עם IP ישן נראה כתקלת חומרה	DHCP ברשת	מזהה `ip_check.py` אי-התאמה בכל עליית בקנד ומזהיר בלוג
ESP32-CAM: איפוס בזמן אתחול וצריבה ידנית מסורבלת	זינוק זרם בהפעלה ומעגל איפוס חסר	ניטרול ה-Brownout Detector, רצף צריבה ידני מתועד ו-FQBN נפרד

6.3 מסקנות

1. **ארכיטקטורה היברידית עובדת.** הפרדה בין בקרים זולים שאחראים על החישה לבין בקנד שאחראי על ההחלטה, הזיהוי והממשק מאפשרת מערכת עשירה ביכולות במחיר חומרה נמוך. בקרי החיישנים נשארו פשוטים, ושינוי מדיניות (כמו מצב Home) לא דרש שינוי קושחה.
2. **נתיב האזעקה הוא הנתיב הקריטי, ולכן נבנה לאי-חסימה.** ההתראה, שידור ה-WebSocket והפעלת הזמזמים אינם חוסמים זה את זה. רוב הבאגים שנמצאו היו מסוג "חסימה", "מצב ננעל" או "כשל שקט", ולא טעויות חישוב.
3. **כשל שקט גרוע מכשל גלוי.** מערכת אזעקה שמציגה מצב תקין כשאינו ידוע מסוכנת יותר מכזו שמציגה שגיאה. לכן כל לוח מדווח חיים, כל פקודה מחזירה אם הגיעה לחומרה, וכל כשל רשת מוצג.
4. **אבטחה היא מדיניות, לא רכיב.** הפרצה החמורה ביותר (נטרול בלי סיסמה) לא נבעה מבאג בקוד אלא מהיעדר החלטה: אף route לא נדרש להגן על עצמו. בדיקה פרמטרית על כל ה-routes גורמת להוספת route לא מוגן להיכשל בבנייה.
5. **בדיקות על חומרה אמיתית אינן ניתנות להחלפה.** ארבע התקלות שנמצאו על השולחן לא היו נתפסות בבדיקה פנימית.
6. **זיהוי פנים הוא נוחות, לא אבטחה.** גם עם שתי בדיקות חיות, הספים לא נמדדו מול זיופים מסודרים, ולכן נטרול פנים נשאר אופציה שכבויה כברירת מחדל.

6.4 מגבלות ידועות

- **תעבורה לא מוצפנת ברשת המקומית.** הלוחות מדברים עם הבקנד ב-HTTP פשוט, ולכן מפתח המכשיר עובר בגלוי ברשת ה-WiFi. ההגנה נשענת על כך שהרשת פרטית, ועל Tailscale לגישה מרחוק.
- **מפתח מכשיר משותף לכל הלוחות.** פגיעה בלוח אחד חושפת את המפתח של כולם, והוא גם המפתח שצופה בזרם הווידאו.
- **סוד משותף יחיד בן 4–8 ספרות** לסיסמת האפליקציה ולקודן, כלומר פחות מ-1.1×10^8 צירופים. זו החלטה מכוונת למערכת ביתית.
- **בקר הדלת משרת לקוח HTTP אחד בכל פעם**, ולכן ניתן בתיאוריה לתפוס את החריץ ולמנוע ממנו לקבל את פקודת הזמזם מהבקנד. ממצא זה נותר פתוח.
- **ללא גיבוי חשמל.** הפסקת חשמל מנתקת את הלוחות; הבקנד מזהה זאת כלוח לא מחובר, אך אינו מונע אותה.
- **`alarm_state` אינו נשמר** בין הפעלות של הבקנד (בכוונה), כך שבקנד שהופעל מחדש עולה כמנוטרל עד לדיווח הבא של בקר הדלת.
- **ללא Push מקורי של FCM** וללא חיוג טלפוני; ההתראות הן ntfy והסלמה בהודעה.
- **דיוק זיהוי הפנים וההשהיות מקצה לקצה** לא נמדדו באופן מסודר (ראה 5.4).
- **Android בלבד**, ומשתמש יחיד לכל מערכת.

6.5 כיווני פיתוח עתידיים

1. **הצפנת התעבורה (HTTPS/TLS)** בין הלוחות לבקנד, ומפתח נפרד לכל לוח, כדי לסגור את שתי המגבלות הראשונות.
2. **נעילה בקודן הפיזי** לאחר שלוש הקשות שגויות, כפי שהוגדר בהצעה המקורית.
3. **כיול מדעי של זיהוי החיות:** איסוף מערך ניסיונות זיוף (הדפסות, מסכים, וידאו), מדידת שיעורי קבלה וסירוב שגויים ובחירת ספים בהתאם, והוספת בדיקה בעלת תכונות מרחביות או אינפרא-אדום.
4. **Push מקורי (FCM)** לאחר יצירת פרויקט Firebase, וחיוג אוטומטי באמצעות ספק טלפוניה.
5. **עמידות בפני הפסקות חשמל:** סוללת גיבוי לבקר הדלת ולבקנד, ושמירת מצב הדריכה כך שיאומת מחדש מול החומרה בהפעלה.
6. **עדכוני קושחה אלחוטיים (OTA)**, שיחסכו צריבה פיזית, במיוחד בלוח המצלמה שצריבתו ידנית.
7. **ריבוי משתמשים והרשאות**, ושימוש בביומטריה של הטלפון לכניסה לאפליקציה.
8. **הרחבת החישה:** חיישני עשן ופריצת זכוכית, חיישן דלת נוסף, ואולטרה-סוני מרובה זוויות.
9. **בדיקות אוטומטיות לקושחה** (למשל הפרדת לוגיקת מכונת המצבים לספרייה הנבדקת על המחשב) וניטור ממושך של יציבות ה-WiFi.
10. **גרסת iOS** והתאמת הממשק לטאבלט.

7. רשימת מקורות

המקורות מובאים לפי כללי IEEE ובסדר הופעתם. הם כוללים דפי נתונים (Datasheets), תיעוד רשמי של ספריות, מאמרים וספרות מקצועית, ותקנים.

דפי נתונים וחומרה

[1] Espressif Systems, "ESP32 Series Datasheet," Espressif Systems, Shanghai, China. [Online]. Available: https://www.espressif.com/sites/default/files/documentation/esp32_datasheet_en.pdf. Accessed: *[תאריך]*.

[2] Elec Freaks, "Ultrasonic Ranging Module HC-SR04," datasheet. [Online]. Available: https://cdn.sparkfun.com/datasheets/Sensors/Proximity/HCSR04.pdf. Accessed: *[תאריך]*.

[3] Texas Instruments, "LM393 Dual Differential Comparator," datasheet. [Online]. Available: https://www.ti.com/product/LM393. Accessed: *[תאריך]*.

[4] Arduino Playground, "Keypad Library." [Online]. Available: https://playground.arduino.cc/Code/Keypad/. Accessed: *[תאריך]*.

ארכיטקטורה, תקשורת ואבטחה

[5] R. T. Fielding, "Architectural styles and the design of network-based software architectures," Ph.D. dissertation, Univ. California, Irvine, CA, USA, 2000.

[6] I. Fette and A. Melnikov, "The WebSocket Protocol," IETF, RFC 6455, Dec. 2011.

[7] Espressif Systems, "HTTP Server," ESP-IDF Programming Guide. [Online]. Available: https://docs.espressif.com/projects/esp-idf/en/latest/esp32/api-reference/protocols/esp_http_server.html. Accessed: *[תאריך]*.

[8] S. Ramírez, "FastAPI Documentation." [Online]. Available: https://fastapi.tiangolo.com/. Accessed: *[תאריך]*.

ראייה ממוחשבת וזיהוי פנים

[9] P. Viola and M. Jones, "Rapid object detection using a boosted cascade of simple features," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Kauai, HI, USA, 2001, pp. I-511–I-518.

[10] W. Wu, H. Peng, and S. Yu, "YuNet: A tiny millisecond-level face detector," *Mach. Intell. Res.*, vol. 20, pp. 656–665, 2023.

[11] Y. Zhong, W. Deng, J. Hu, D. Zhao, X. Li, and D. Wen, "SFace: Sigmoid-constrained hypersphere loss for robust face recognition," *IEEE Trans. Image Process.*, vol. 30, pp. 2587–2598, 2021.

[12] OpenCV, "DNN-based Face Detection And Recognition," OpenCV Documentation. [Online]. Available: https://docs.opencv.org/4.x/d0/dd4/tutorial_dnn_face.html. Accessed: *[תאריך]*.

אבטחת מידע, רשת והתראות

[13] C. Percival and S. Josefsson, "The scrypt Password-Based Key Derivation Function," IETF, RFC 7914, Aug. 2016.

[14] J. A. Donenfeld, "WireGuard: Next generation kernel network tunnel," in *Proc. Netw. Distrib. Syst. Secur. Symp. (NDSS)*, San Diego, CA, USA, 2017.

[15] Tailscale, "How Tailscale works." [Online]. Available: https://tailscale.com/blog/how-tailscale-works. Accessed: *[תאריך]*.

[16] ntfy, "ntfy Documentation." [Online]. Available: https://docs.ntfy.sh/. Accessed: *[תאריך]*.

אפליקציה

[17] Google, "Flutter Documentation." [Online]. Available: https://docs.flutter.dev/. Accessed: *[תאריך]*.

[18] Minivision AI, "Silent-Face-Anti-Spoofing," GitHub repository. [Online]. Available: https://github.com/minivision-ai/Silent-Face-Anti-Spoofing. Accessed: *[תאריך]*.

קושחה — תיעוד המערכת והספריות

[19] Espressif Systems, "Non-Volatile Storage Library," ESP-IDF Programming Guide. [Online]. Available: https://docs.espressif.com/projects/esp-idf/en/latest/esp32/api-reference/storage/nvs_flash.html. Accessed: *[תאריך]*.

[20] Espressif Systems, "esp32-camera: ESP32 camera driver," GitHub repository. [Online]. Available: https://github.com/espressif/esp32-camera. Accessed: *[תאריך]*.

[21] FreeRTOS, "Queue Management." [Online]. Available: https://www.freertos.org/Embedded-RTOS-Queues.html. Accessed: *[תאריך]*.

[22] Encode, "HTTPX Documentation." [Online]. Available: https://www.python-httpx.org/. Accessed: *[תאריך]*.

[23] R. Rousselet, "provider," pub.dev package. [Online]. Available: https://pub.dev/packages/provider. Accessed: *[תאריך]*.

[24] Espressif Systems, "Arduino-ESP32 Documentation." [Online]. Available: https://docs.espressif.com/projects/arduino-esp32/en/latest/. Accessed: *[תאריך]*.

[25] Arduino, "attachInterrupt()," Arduino Language Reference. [Online]. Available: https://www.arduino.cc/reference/en/language/functions/external-interrupts/attachinterrupt/. Accessed: *[תאריך]*.

ספריות בקנד, כלי פיתוח ובדיקה

[26] Encode, "Uvicorn: An ASGI web server for Python." [Online]. Available: https://www.uvicorn.org/. Accessed: *[תאריך]*.

[27] Pydantic, "Pydantic Documentation." [Online]. Available: https://docs.pydantic.dev/. Accessed: *[תאריך]*.

[28] C. R. Harris *et al.*, "Array programming with NumPy," *Nature*, vol. 585, pp. 357–362, 2020.

[29] G. Bradski, "The OpenCV Library," *Dr. Dobb's J. Softw. Tools*, 2000.

[30] OpenCV, "OpenCV Zoo: a model zoo for OpenCV DNN," GitHub repository. [Online]. Available: https://github.com/opencv/opencv_zoo. Accessed: *[תאריך]*.

[31] pytest-dev, "pytest Documentation." [Online]. Available: https://docs.pytest.org/. Accessed: *[תאריך]*.

[32] GitHub, "GitHub Actions Documentation." [Online]. Available: https://docs.github.com/en/actions. Accessed: *[תאריך]*.

[33] Arduino, "Arduino CLI Documentation." [Online]. Available: https://arduino.github.io/arduino-cli/. Accessed: *[תאריך]*.

חבילות Flutter וממשק משתמש

[34] Dart team, "web_socket_channel," pub.dev package. [Online]. Available: https://pub.dev/packages/web_socket_channel. Accessed: *[תאריך]*.

[35] Flutter team, "shared_preferences," pub.dev package. [Online]. Available: https://pub.dev/packages/shared_preferences. Accessed: *[תאריך]*.

[36] flutter_local_notifications," pub.dev package. [Online]. Available:" https://pub.dev/packages/flutter_local_notifications. Accessed: *[תאריך]*.

[37] Flutter team, "google_fonts," pub.dev package. [Online]. Available: https://pub.dev/packages/google_fonts. Accessed: *[תאריך]*.

[38] Dart team, "http," pub.dev package. [Online]. Available: https://pub.dev/packages/http. Accessed: *[תאריך]*.

[39] Google, "Material Design 3." [Online]. Available: https://m3.material.io/. Accessed: *[תאריך]*.

8. נספחים

נספח א' — רשימת חומרים (BOM)

טבלה 18: רשימת חומרים מלאה

#	שם הרכיב	תיאור / תפקיד במערכת	כמות	מחיר (₪)	קישור לרכישה
1	ESP32 DevKit V1 (ESP32-WROOM-32)	בקר הקצה של שלושה לוחות: אולטרה-סוני, רעידות, ובקר הדלת והקודן. WiFi מובנה, מריץ את הקושחה ב-C++	3	*[להשלים]*	[הוסף קישור לרכישה]
2	ESP32-CAM (AI-Thinker, חיישן OV2640)	לוח המצלמה: דוחף פריימים לזיהוי פנים ומגיש זרם MJPEG חי. מגיע עם 4 MB PSRAM	1	*[להשלים]*	[הוסף קישור לרכישה]
3	חיישן מרחק אולטרה-סוני HC-SR04	ניטור היקפי: זיהוי כניסה או מעבר באזור מוגדר על פי זמן הלוך-חזור של גל קול	1	*[להשלים]*	[הוסף קישור לרכישה]
4	מודול חיישן רעידות SW-420 (עם משווה LM393)	זיהוי פריצה בכוח (מכה בדלת או בחלון): אחד בלוח הרעידות, ואחד בלוח האולטרה-סוני לחלון סמוך	2	*[להשלים]*	[הוסף קישור לרכישה]
5	חיישן דלת מגנטי MC-38 (Reed Switch)	זיהוי פתיחת הדלת הראשית; מחובר לבקר הדלת	1	*[להשלים]*	[הוסף קישור לרכישה]
6	קודן מטריצה 3x4	הזנת קוד לדריכה ולנטרול פיזיים; 7 פינים (4 שורות, 3 עמודות)	1	*[להשלים]*	[הוסף קישור לרכישה]
7	זמזם (Buzzer)	אזעקה קולית ואזהרה מואצת בחלון הנטרול; שני זמזמים המופעלים לסירוגין בבקר הדלת *[לאמת: פעיל / פסיבי]*	2	*[להשלים]*	[הוסף קישור לרכישה]
8	כבלי USB ומטען 5V	הזנת ארבעת הלוחות וצריבת הקושחה	*[לאמת]*	*[להשלים]*	[הוסף קישור לרכישה]
9	ממיר USB-Serial (FTDI / CP2102 או בסיס צריבה ל-ESP32-CAM	צריבת לוח המצלמה, שאין בו מחבר USB *[לאמת אם נעשה שימוש]*	1	*[להשלים]*	[הוסף קישור לרכישה]

#	שם הרכיב	תיאור / תפקיד במערכת	כמות	מחיר (₪)	קישור לרכישה
10	לוחות ניסוי (Breadboard), כבלי חיבור (Jumpers), נגדים	חיווט הרכיבים והמעגלים	*[לא מתם]*	*[להשלים]*	[הוסף קישור לרכישה]
11	*(אופציונלי)* מצלמת USB	חלופה ל-ESP32-CAM דרך `camera_worker.py`; אינה נדרשת	0–1	*[להשלים]*	[הוסף קישור לרכישה]
	סה"כ			*[להשלים]* מתוך תקציב של 170 ₪ בהצעה	

הערה: ברשימת ההצעה המקורית הופיעו גם חיישן ומשדר לייזר ומצלמת USB; הם הוחלפו בחיישן האולטרה-סוני ובמודול ה-ESP32-CAM (ראה טבלה 2).

נספח ב' — מבנה הקבצים המצורפים

הקוד המלא והקבצים הנלווים מאורגנים כך:

תיקייה	תוכן
`/esp32`	ארבעה sketches: `ultrasonic_sensor`, `vibration_sensor`, `door_keypad_controller`, `esp32_cam`; התבנית `secrets.h.example`
`/python`	הבקנד (`/backend`), `simulate.py`, `camera_worker.py`, הבדיקות (`/tests`), מודלי ה-ONNX (`/models`) ו-`requirements.txt`
`/app`	אפליקציית Flutter: `lib/` (מסכים, שירותים, ווידג'טים, ערכת עיצוב), `test/`, ונכסי הגופנים
`/docs`	טופס הצעת הפרויקט המקורי
`/scripts`	סקריפטי התקנת סביבת הפיתוח (Windows ו-WSL2)
`.github/workflows/`	הגדרת האינטגרציה הרציפה

[הכנס כאן קישור לסרטון הדגמת המערכת (כפי שצוין בסוף המבוא), וקישור למאגר הקוד אם רלוונטי]

הקוד המלא והקבצים הנלווים מצורפים בקובץ ה-ZIP של הגשת הפרויקט.